LAB#0



– Digital Electronics –
Digital design flow with Quartus-Prime

This tutorial introduces the Quartus-Prime CAD system. It presents a general overview of a typical CAD flow for designing circuits to be implemented in FPGAs. The design process is illustrated by step-by-step instructions regarding the complete flow to implement a very simple circuit in an Altera FPGA device. The Quartus Prime software system includes full support for all of the popular steps about the description of a circuit in a CAD system. This tutorial presents the Verilog/VHDL design entry method, in which the user describes the hardware with the Verilog or with the VHDL description language.

The last step in the design process involves the configuration of the designed circuit in an actual FPGA device. To show how this is done, it is assumed that the user can access an Altera DE1-SoC Development and Education board connected to a computer that has the Quartus Prime software installed. A reader who does not have access to the DE1-SoC board will still find the tutorial useful to learn how to program an FPGA and how the configuration task is performed.

The screen captures in the document are obtained by using the Quartus Prime version 16.1; if other versions of the software are used, some of the screenshots may be slightly different.

Contents:

I. Typical CAD flow
1. Getting started
2. Starting a New Project
3. HDL Design Entry
4. Compiling the Design
5. Pin Assignment
6. Simulating the Circuit
7. Programming and Configuring the FPGA
8. Testing the Designed Circuit

Appendix:
A. Schematic Design Entry

Abbreviations and acronyms:

AS – Active Serial mode
ASCII – American Standard Code for Information Interchange
CAD – Computer Aided Design
EDA – Electronics Design Automation
FPGA – Field Programmable Gate Array
GUI – Graphical User Interface
IC – Integrated Circuit
JTAG – Joint Test Action Group
LE – Logic Element
LED – Light Emitting Diode
MUX – Multiplexer

MAKE SURE YOU UNDERSTAND EVERYTHING. IF NOT, YOU WILL NOT BE ABLE TO DO THE NEXT LAB SESSIONS!
IF SOMETHING IS NOT CLEAR ASK THE TEACHER.

HOMEWORK: study and try this part on a PC, you do not need the DE1-SoC board to complete this part

I – Typical CAD flow

Computer Aided Design (CAD) software aims at simplifying logic circuits designs and implementations. Part of the CAD software available for commercial use aims at programming flexible devices such as Field-Programmable Gate Arrays (FPGAs). In this lab session, we will focus on a typical digital design flow, which last step is the programming of an FPGA device. A typical FPGA-CAD flow is illustrated in Figure 1.

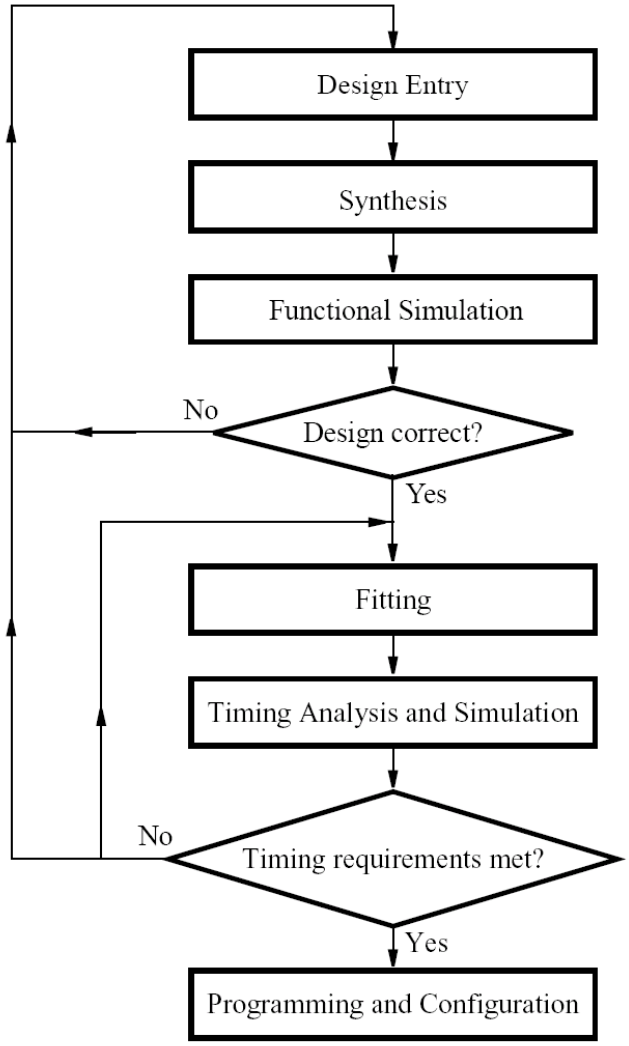


Figure 1 - Typical FPGA-CAD design flow.

The CAD design flow involves the following steps:

• **Design Entry** – The circuit is specified as a Register Transfer Level (RTL) description by means of either a Verilog or a VHDL description.
• **Synthesis** – The entered design is synthesized into a circuit that consists of Logic Elements (LEs) provided by the FPGA IC.

- **Functional Simulation** – The circuit (either RTL or synthesized) is tested to verify its functional correctness; this simulation does not take into account any timing issues.
- **Fitting** – The CAD Fitter tool determines the placement of the LEs defined in the netlist into the real LEs of an actual FPGA chip; it also chooses routing wires in the chip to define the required connections between specific LEs. Other kinds of configurable logic components available in the selected FPGA may be exploited by the tool.
- **Timing Analysis** – Propagation delays along the various paths are also analyzed to estimate the expected performance of the circuit.
- **Timing Simulation** – The synthesized circuit is tested to verify both its functional correctness and if it meets the timing requirements.
- **Programming and Configuration** – The designed circuit is implemented in a physical FPGA chip by programming the configuration switches to route signals to the LEs and by establishing the required wiring connections.

This tutorial introduces the basic features of the Quartus Prime software. It shows how the software is used to design and implement a circuit specified by means of an HDL description. To speed-up the design process the software provides a Graphical User Interface (GUI) that invokes the commands at the lower software levels.

After completing this tutorial, you will learn how to:

- *Create a project*
- *Enter an HDL design*
- *Synthesize a circuit from the HDL description*
- *Dump a synthesized circuit into an Altera FPGA*
- *Assign the circuit inputs and outputs to specific pins on the FPGA*
- *Simulate the designed circuit with realistic FPGA models*
- *Program and configure the FPGA chip on Altera DE1-SoC board*

Keep in mind that the next laboratory sessions will deal with digital HDL design, thus the design entry method used in the main part of this tutorial is fundamental. Do this tutorial step-by-step and try to learn this procedure efficiently in such a way that you can speed-up the execution of the next labs thus saving time!

1 - Getting Started

Each logic circuit, or sub-circuit being designed with Quartus Prime software is called a *project*. The software works on one project at a time and keeps all the information of the open project in a single directory (folder) on the file-system.

To begin a new logic circuit design, the first step is to create a directory to hold its files. In our case, we will create a directory called *introtutorial*. The name of the directory should refer to the aim of the project. The example described in this tutorial is a simple circuit for a two-way light controller.

Start Quartus Prime.. You should now see a display similar to the one in Figure 2. This display consists of several windows that provide access to all the functionalities of the software, which the user can select with the help of the computer mouse. Most of the commands provided by Quartus Prime can be run by using a set of menus that are located below the title bar. For example, by referring to the screenshot of Figure 2, by clicking with the left mouse button on the menu called File it is possible to show the menu in Figure 3. By clicking the left mouse button on the entry Exit the user terminates the execution of Quartus Prime. In general, the *left* button is the default for most of the commands hence we will specify the mouse button only in the cases in which it is necessary to use the *right* mouse button.

For some commands, it is necessary to access two or more menus in sequence. We use the convention Menu1 > Menu2 > Item to indicate that the selection of the command action is done by first clicking the left mouse button on Menu1, then within this menu clicking on Menu2. Again, within Menu2 the command is executed by clicking on Item. For example, File > Exit indicates to click on the exit command of Quartus Prime by using the mouse. Many commands can be invoked by clicking on the corresponding icon displayed in one of the toolbars. To see the command associated with an icon, position the mouse over the icon; soon a tool-tip will appear displaying the command name.

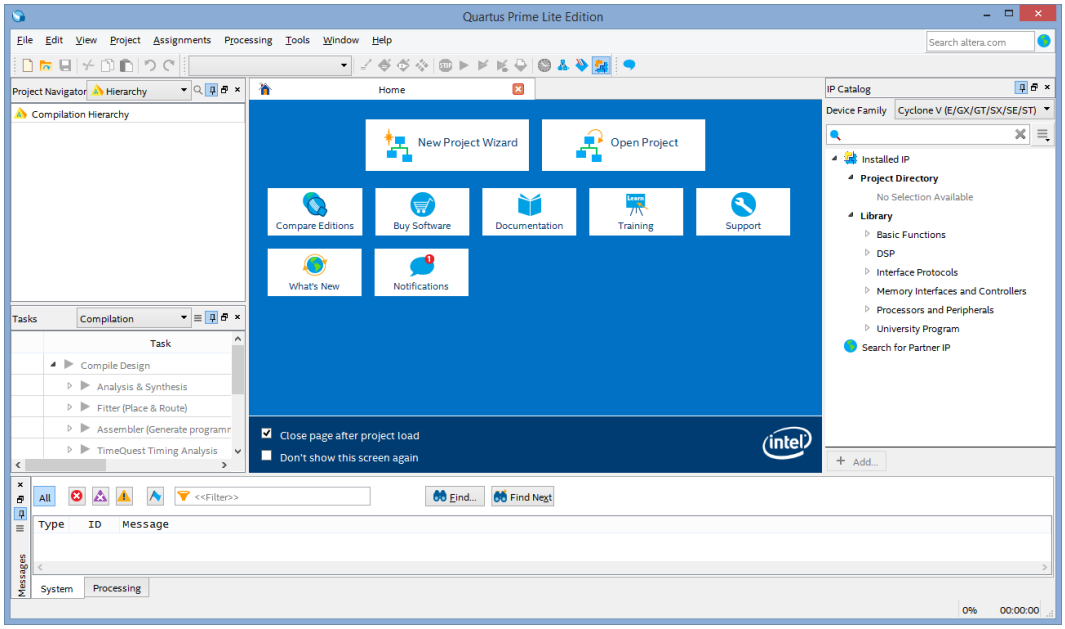


Figure 2 - The main Quartus Prime main window.

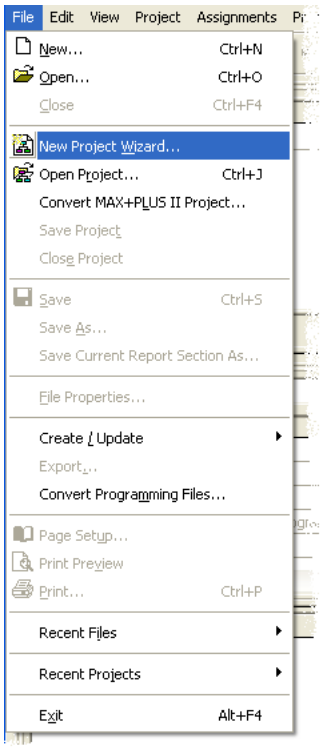


Figure 3 - File menu.

1.1 - Quartus Prime Online Help

Quartus Prime provides comprehensive online documentation answering many of the questions that may arise when using the software. The documentation is accessed from the Help menu. To have an idea of the extent of the

documentation, it is worthwhile for the reader to browse through the Help menu. For instance, by selecting Help > How to Use Help it is possible to have an indication of what type of help documentation is included in the software.

The user can quickly search through the Help topics by selecting Help > Search which opens a dialog box into which key words can be entered. Another method, based on context-sensitive help, can be followed for quickly finding documentation about specific topics. While using most applications under the Windows operating system, the F1 function key on the keyboard normally opens a Help window, which shows the available commands of the application or a brief explanation of the textboxes if the window is a common dialog.

2 - Starting a New Project

To start working on a new design we first have to define a new *design project*. The creation of a new project is very easy thanks to a *wizard*. Create a new project as follows:

1. Select File > New Project Wizard to open the window in Figure 4. It indicates the main capabilities of this wizard. You can skip this window in subsequent projects by checking the box Don't show me this introduction again. Press Next to get the window shown in Figure 5. As an alternative option, you can also open the wizard directly from the main Quartus window.

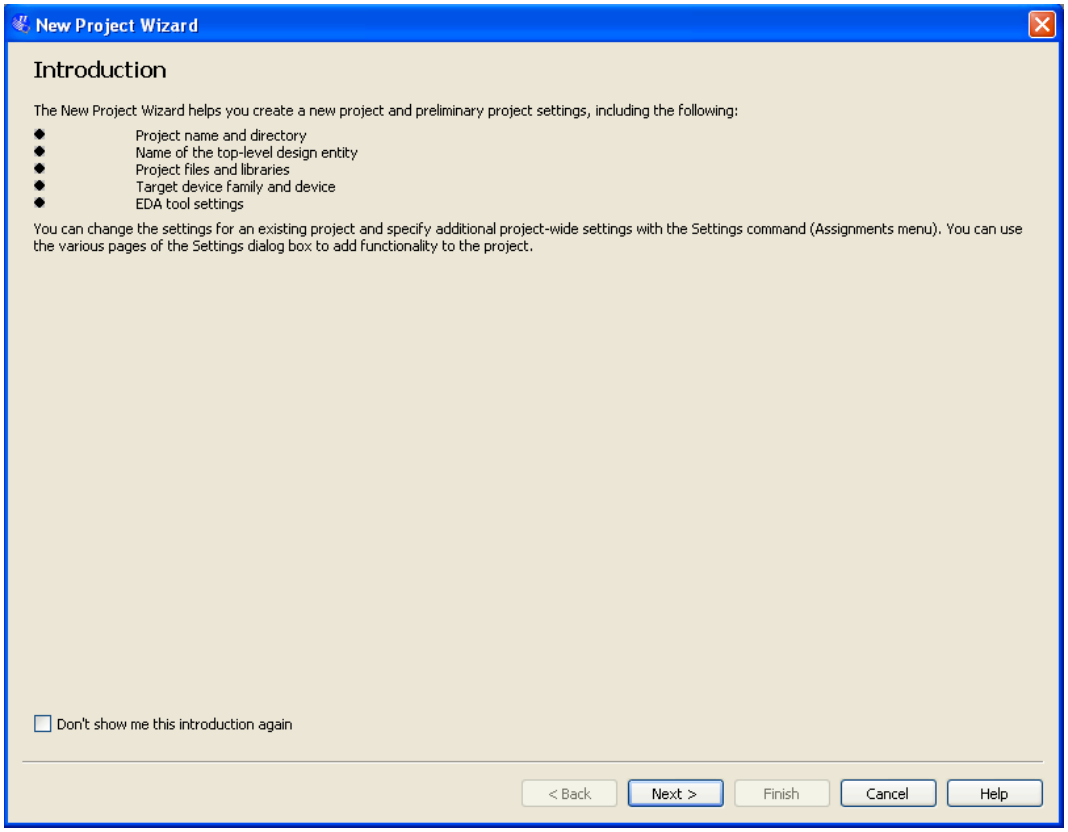


Figure 4 – Possible tasks the wizard can complete.

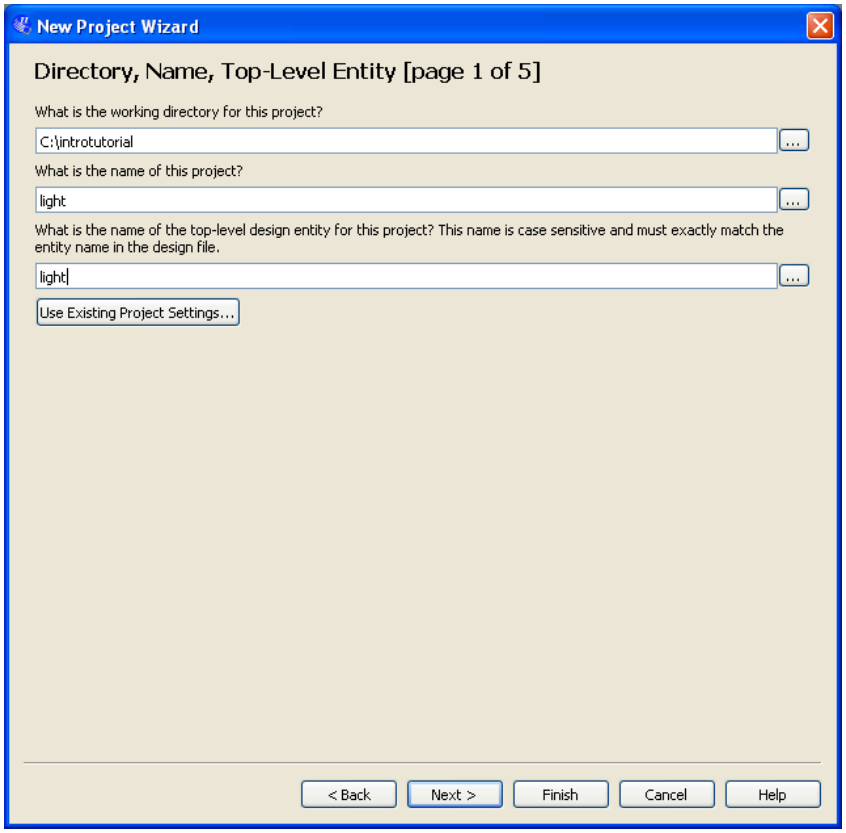


Figure 5 - Creating a new project.

2. Set the working directory to be *introtutorial*; of course, you can use some other directory name if you prefer. Be sure to work in a directory where your user has writing rights. The project must have a name, which is usually the same as the top-level design entity that will be included in the project. Choose *light* as the name for both the project and the top-level entity as shown in Figure 5. Press Next. Since we have not yet created the directory *introtutorial*, Quartus Prime displays a pop-up box to ask if the directory can be created. Click on Yes. This will lead to Figure 6.

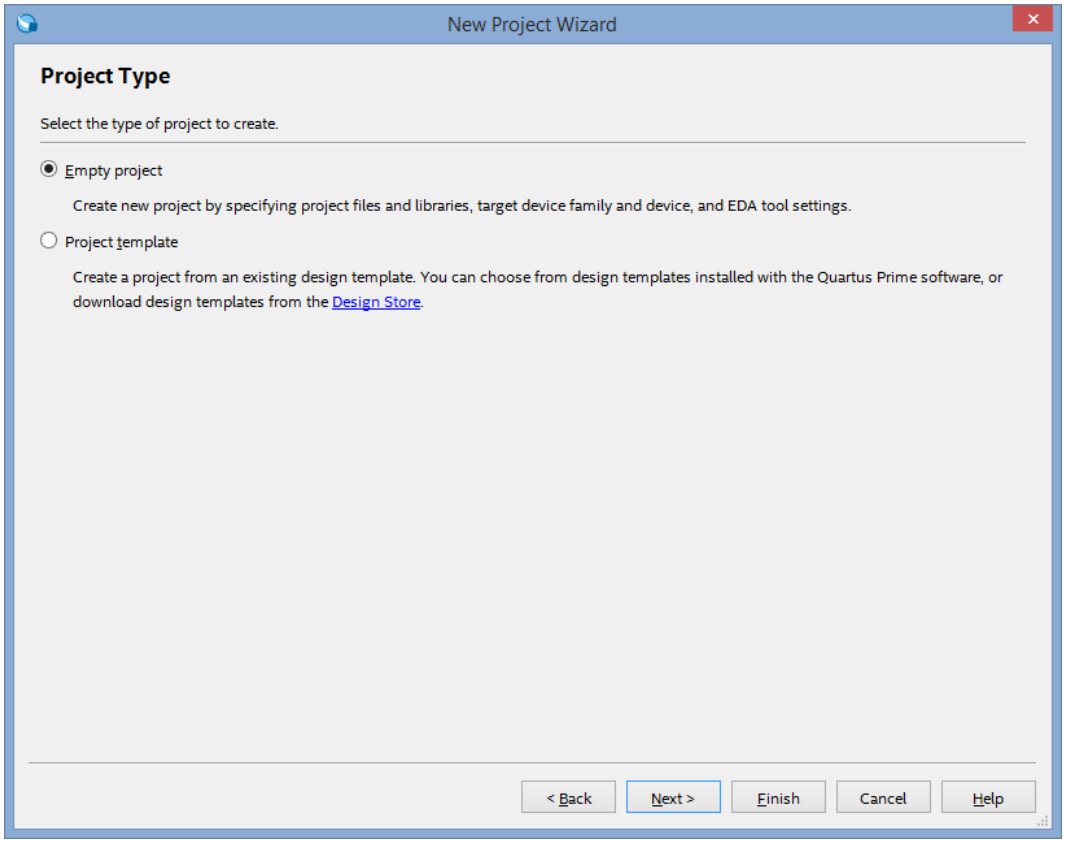


Figure 6 – Choosing the project type..

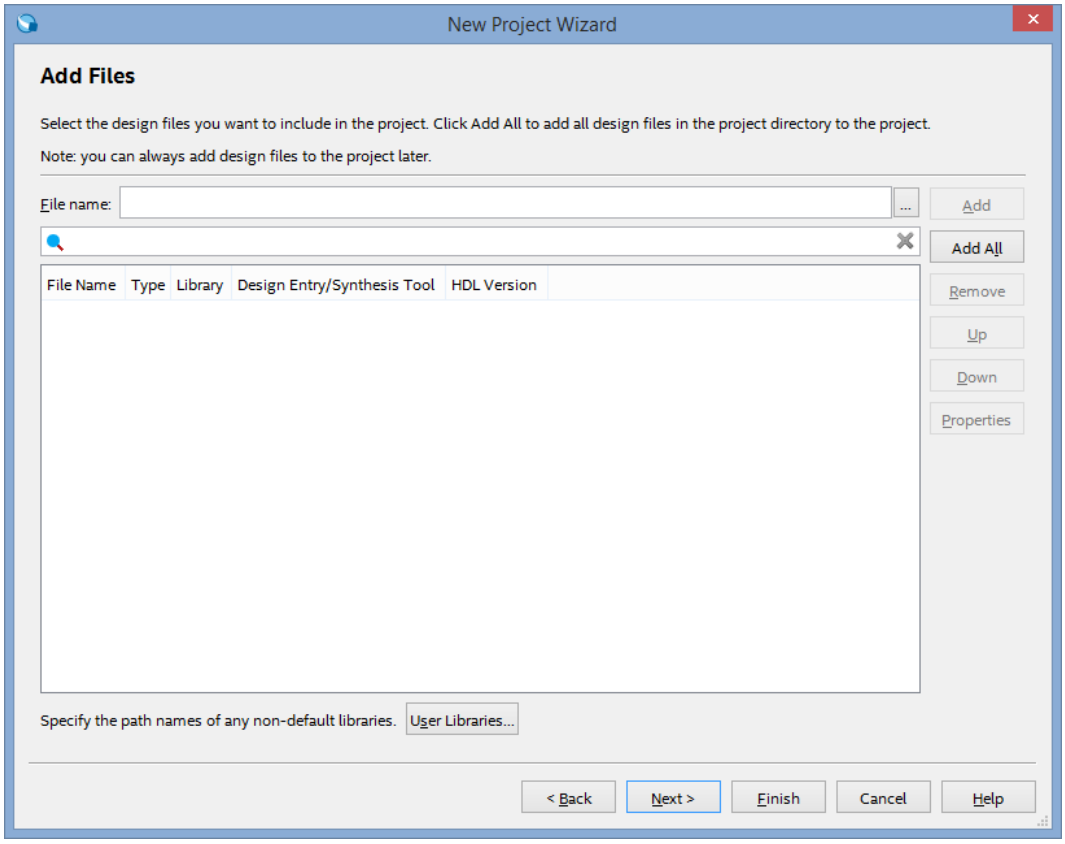


Figure 7 - The wizard can include user-specified design files.

3. The Project Type window, shown in Figure 6, allows you to choose from the Empty project and the Project template options. For this tutorial, choose Empty project as we will be creating a project from scratch, and press Next which leads to the window in Figure 7.
4. With this window, the wizard is asking to include some pre-existing design files in the project. We assume we do not have any, thus we can click on Next. Figure 8 should pop-up.

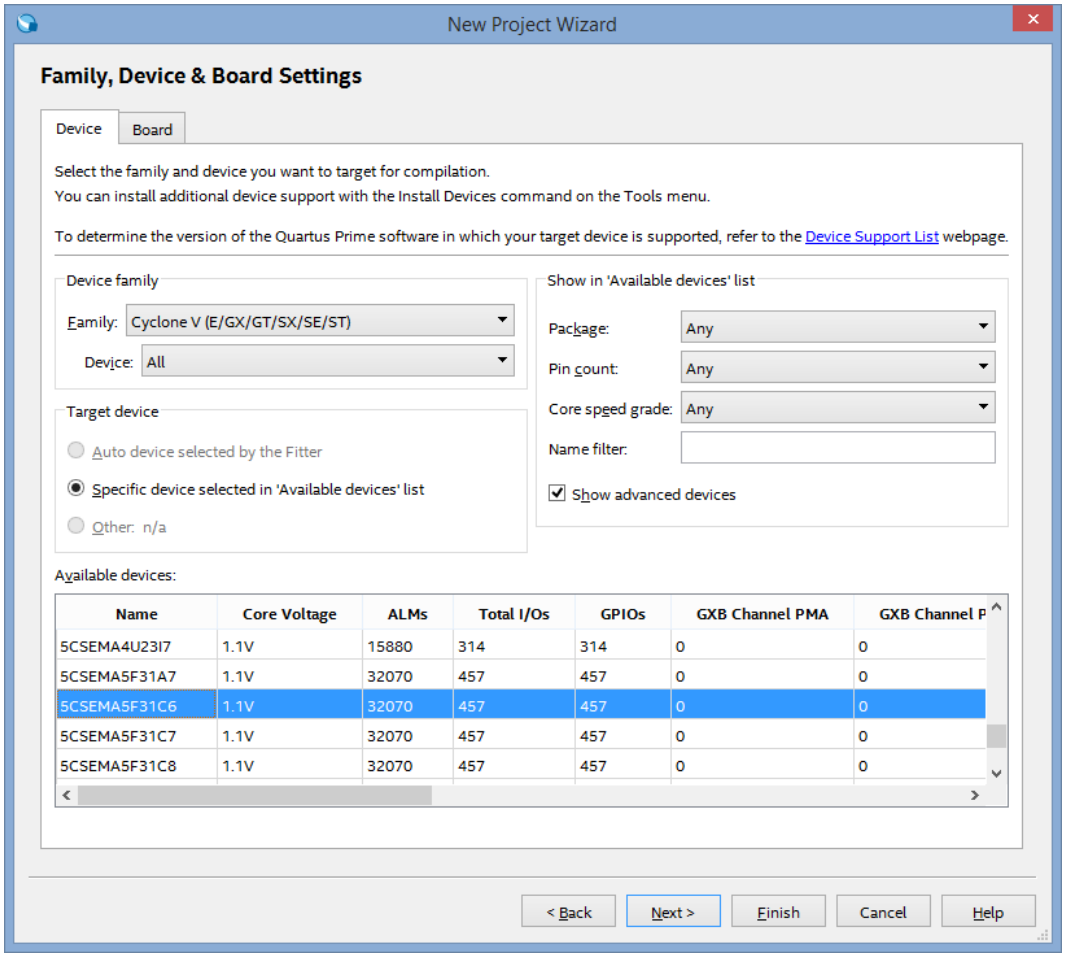


Figure 8 - Device family and specific devices.

5. We have to specify the type of device in which the designed circuit will be implemented. Choose Cyclone V as the target device family. We can let Quartus Prime software select a specific device in the family, or we can choose the device explicitly. We will take the latter approach. From the list of available devices, choose the device called 5CSEMA5F31C6 which is the FPGA used on Altera's DE1-SoC board. Press on Next. Figure 9 should be displayed.
6. The user can specify any third-party tools used in the project. For this purpose, a common term for electronics CAD softwares is *EDA tools*, where the acronym stands for Electronic Design Automation. This term is used in Quartus Prime messages to refer to third party tools, which are software developed and marketed by different companies than Altera. Note: if you already installed ModelSim or QuestaSim Altera FPGA Startet, then Quartus will propose it for simulation. Press Next.
7. A summary of the project settings appears in the screenshot shown in Figure 10. Press Finish. Now the software should return to the main Quartus Prime window. Now the title bar should specify the name of the project that, as previously mentioned, is *light* (Figure 11).

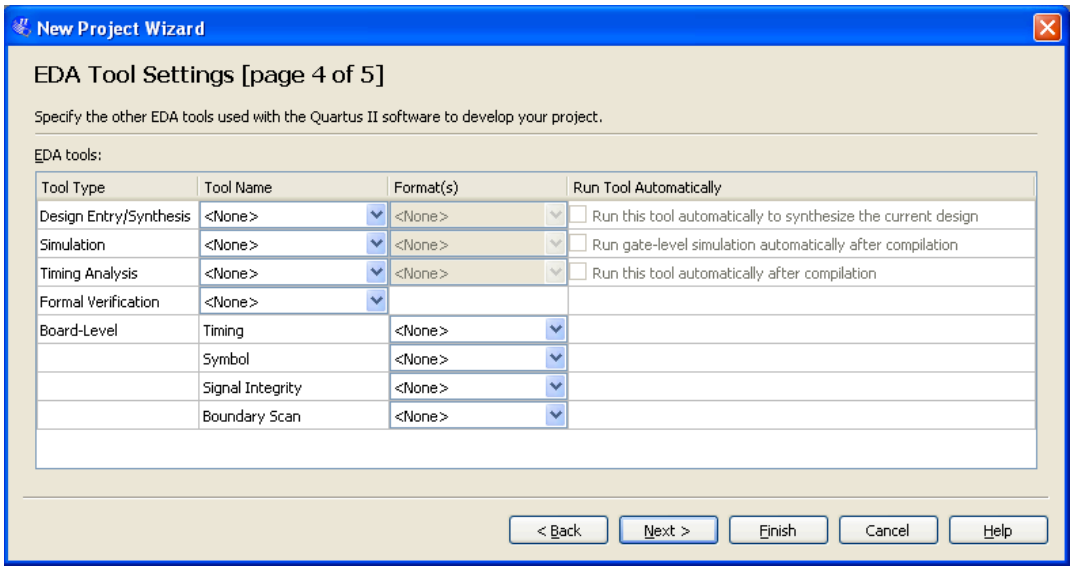


Figure 9 - Other EDA tools can be specified.

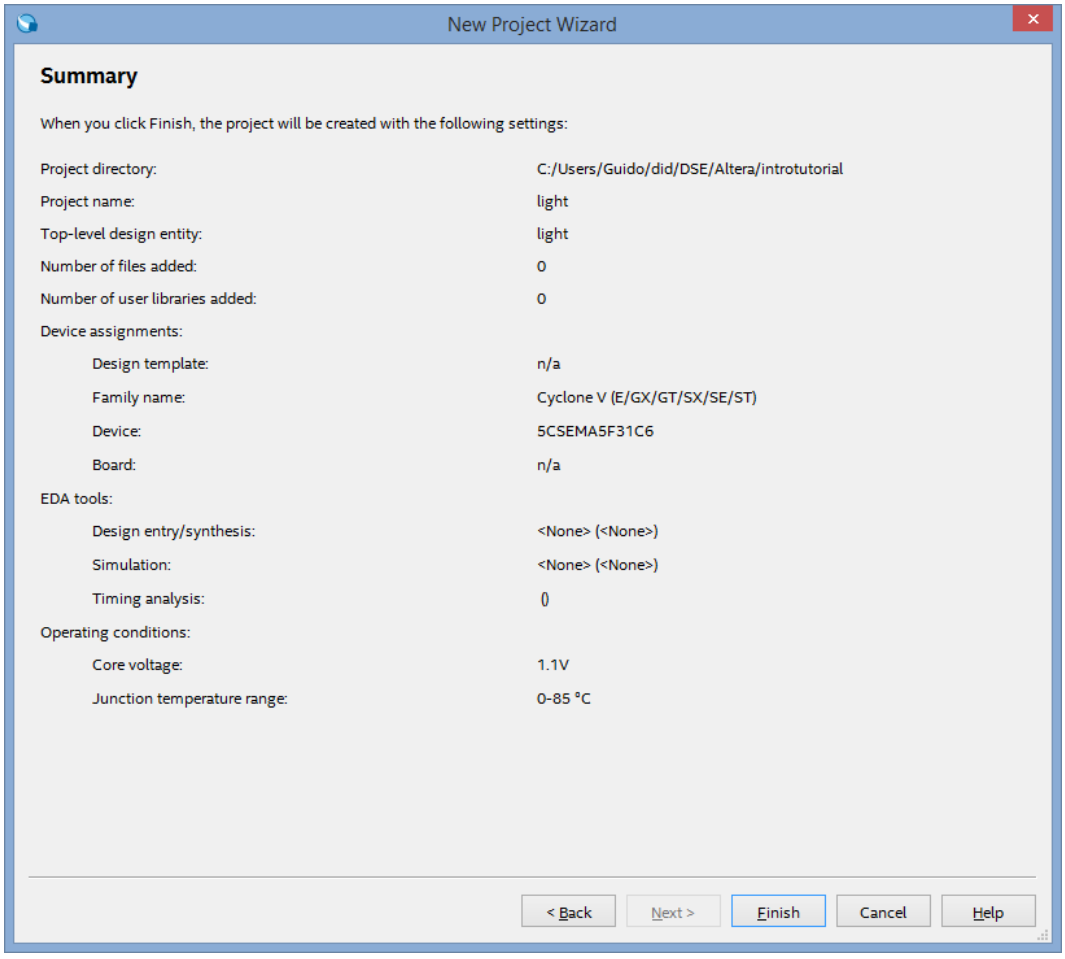


Figure 10 - Summary of the project settings.

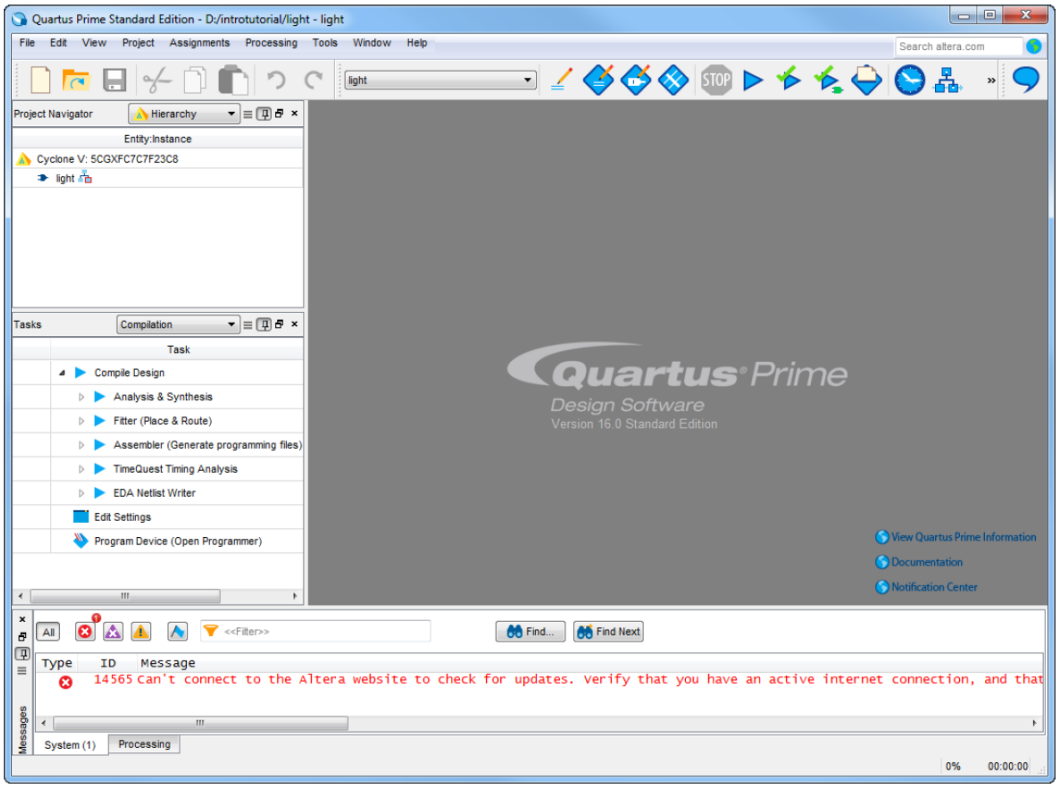
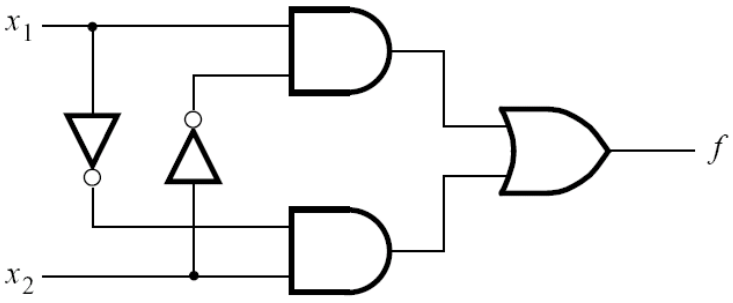


Figure 11 - The Quartus Prime main window customized to the created project.

3 –HDL Design Entry

As a design example, we will use the two-way light controller circuit shown in Figure 12. The circuit can be used to control a single light from either the two switches x_1 and x_2 where a closed switch corresponds to the logic value 1. The truth table of the circuit is also given in the figure. Note that this is very simple and consists of the Exclusive-OR of the inputs x_1 and x_2.



x_1	x_2	f
0	0	0
0	1	1
1	0	1
1	1	0

Figure 12 - The light controller circuit.

As you are learning in the class, a possible code for describing the circuit could be the one shown in Figure 13. Note that the HDL module/entity is called *light* to match the name given at the project creation step. This code can be typed and stored in a file by using any ASCII text editor, or by using the Quartus Prime text editing facilities. Even if it is not mandatory, it is recommended that the same name is used for both module/entity and file containing the module/entity. The

file name must include the proper extension (v or vhd), which indicates that the text file is a Verilog or a VHDL file. Here we will use the name *light.v/light.vhd*.

```
// Verilog
module light (
    input  x1,
    input  x2,
    output f
);
    assign f = (x1 & ~x2) | (~x1 & x2);
endmodule



-- VHDL
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY light IS
PORT ( x1, x2 : IN  STD_LOGIC ;
            f : OUT STD_LOGIC ) ;
END light ;

ARCHITECTURE LogicFunction OF light IS
BEGIN
  f <= (x1 AND NOT x2) OR (NOT x1 AND x2);
END LogicFunction ;
```

Figure 13 – Verilog and VHDL code for the circuit in Figure 12.

3.2 - Adding Design Files to a Project

Now you can tell Quartus Prime software which design files it should use as part of the current project. To see the list of files already included in the *light* project, select Assignments > Settings. Figure 14 should appear. As indicated on the left side of the figure, click on the item Files. An alternative way of making this selection is to choose the menu Project > Add/Remove Files in Project.

If you used the Quartus Prime Text Editor to create the file and checked the box labeled Add file to current project, then the *light.v/light.vhd* file is already a part of the project and will be listed in the window in Figure 14. Otherwise, the file must be added to the project. So, if you did not use the Quartus Prime Text Editor, then copy the file *light.v/light.vhd* to the directory *introtutorial*. To add this file to the project, click on the File name: … button in Figure 14 and get the pop-up window in Figure 15. Select the *light.v/ligth.vhd* file and click Open. The selected file is now indicated in the Files window of Figure 14. Click OK to include the *light.v/light.vhd* file in the project. We should mention that in many cases Quartus Prime is able to automatically find the right files for each module/entity written in the Verilog/VHDL code, even if the file has not been explicitly added to the project. However, for complex projects that involve many files it is a good practice to specifically add the files to the project as described above.

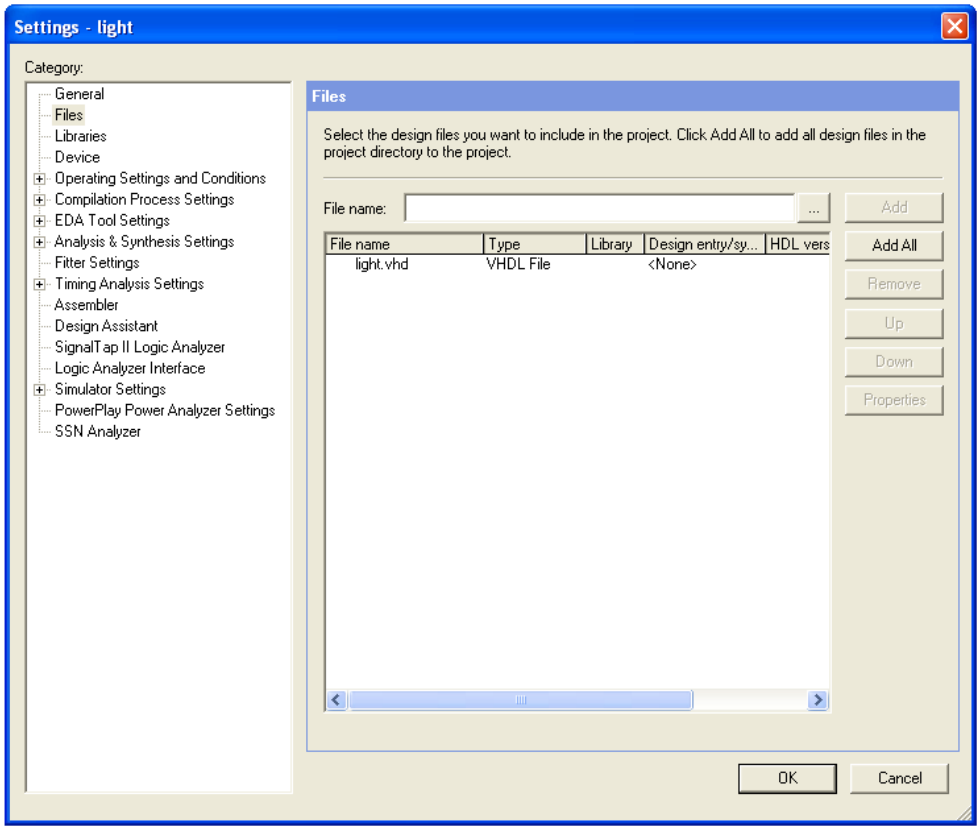


Figure 14 - Settings window.

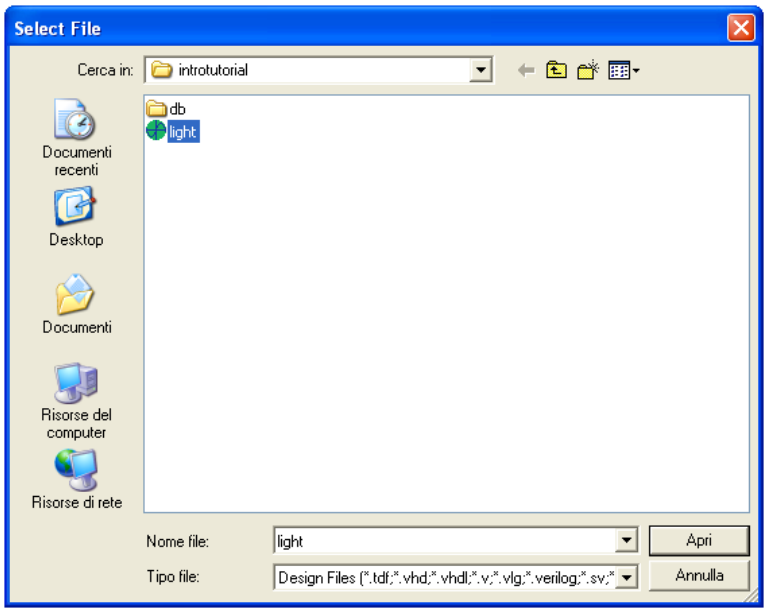


Figure 15 - Select the file.

4 - Compiling the Designed Circuit

The entered HDL file is processed by several Quartus Prime sub-parts that analyze the file, synthesize the circuit, and implement it onto the FPGA. These tools can be invoked and are controlled by the application called the *Compiler*. Run the Compiler by selecting Processing > Start Compilation, or by clicking on the toolbar the compile icon that looks like a light blue triangle. Your project must be saved before compiling. As the compilation moves through various stages, its progress is reported in a window on the left side of the Quartus Prime display. In the message window, at the bottom of the figure, various messages are displayed throughout the compilation process. In case of errors, there will be appropriate messages given. When the compilation is finished, a compilation report is produced. A tab showing this report is opened automatically, as seen in Figure 16. The report can be opened at any time by selecting Processing > Compilation Report. The report includes a number of sections listed on the left-hand-side of the window. Figure 16 displays the Compiler Flow Summary section, which indicates in this case that only one logic element and three pins are needed to implement this tiny circuit on the selected FPGA chip. By clicking on Analysis & Synthesis and then on Resource Usage Summary it is possible to obtain other information about logic elements used for synthesis (figure 17).

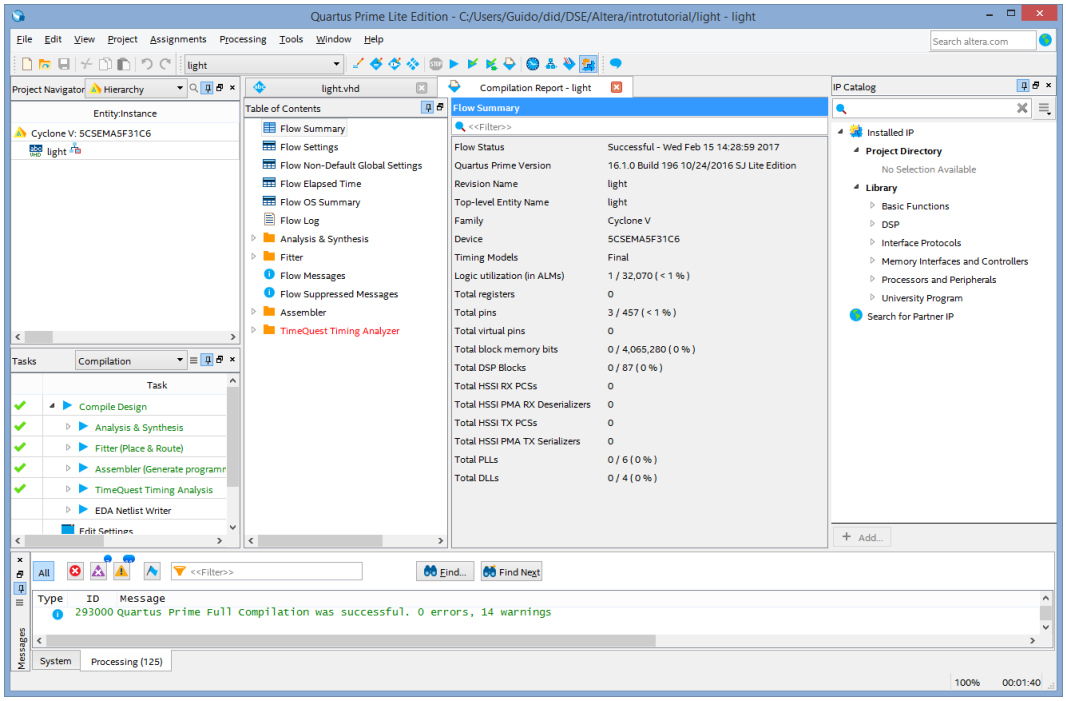


Figure 16 - Display after a successful compilation.

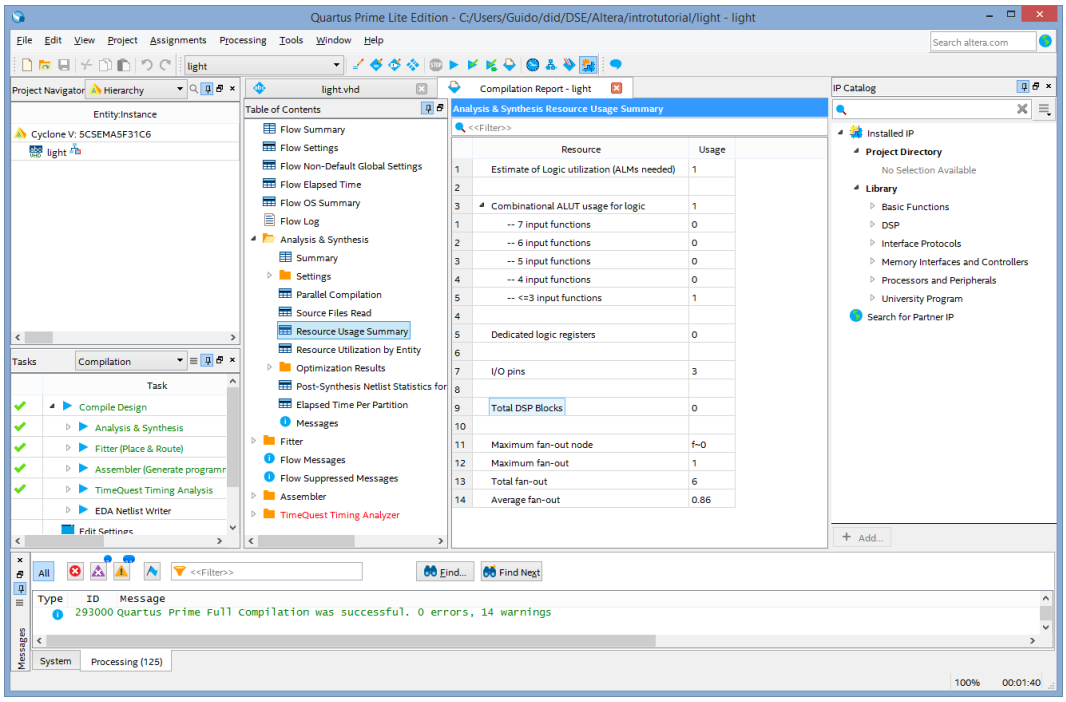


Figure 17 – Resource usage summary.

4.1 - Errors

Quartus Prime software displays messages produced during compilation in the Messages window. If the HDL design file is correct, one of the messages will state that the compilation was successful and that there are no errors. If the Compiler does not report zero errors, then there is at least one mistake in the HDL code. In this case, a message corresponding to each error will be displayed in the Messages window with precise information about the line in your HDL which generated the error. By double-clicking on an error message, the offending statement in the HDL code will be highlighted in the Text Editor window. Similarly, the Compiler may display some warning messages. Their details can be viewed in the same way as in the case of error messages. The user can obtain more information about a specific error or warning message by selecting the message and pressing the F1 function key.

To see the effect of an error, open the file *light.v/light.vhd*. remove the semicolon in the statement that defines the function assigned to output `f`, thus adding a typographical error that can easily occur. Compile the erroneous design file by clicking on the compile icon. A pop-up box will ask if the changes made to the *light.v/light.vhd* file should be saved; click Yes. After having compiled the model, Quartus Prime software will display a pop-up box indicating that the compilation was not successful. Acknowledge it by clicking OK. The compilation report summary, given in Figure 18, now confirms the failed compilation. Select Messages at the bottom of the window to have all the compilation output displayed as shown in Figure 19. Double-click on the first error message. Quartus Prime opens *light.v/light.vhd* and highlights the statement that is affected by the error, as shown in Figure 20. Correct the error and recompile the design.

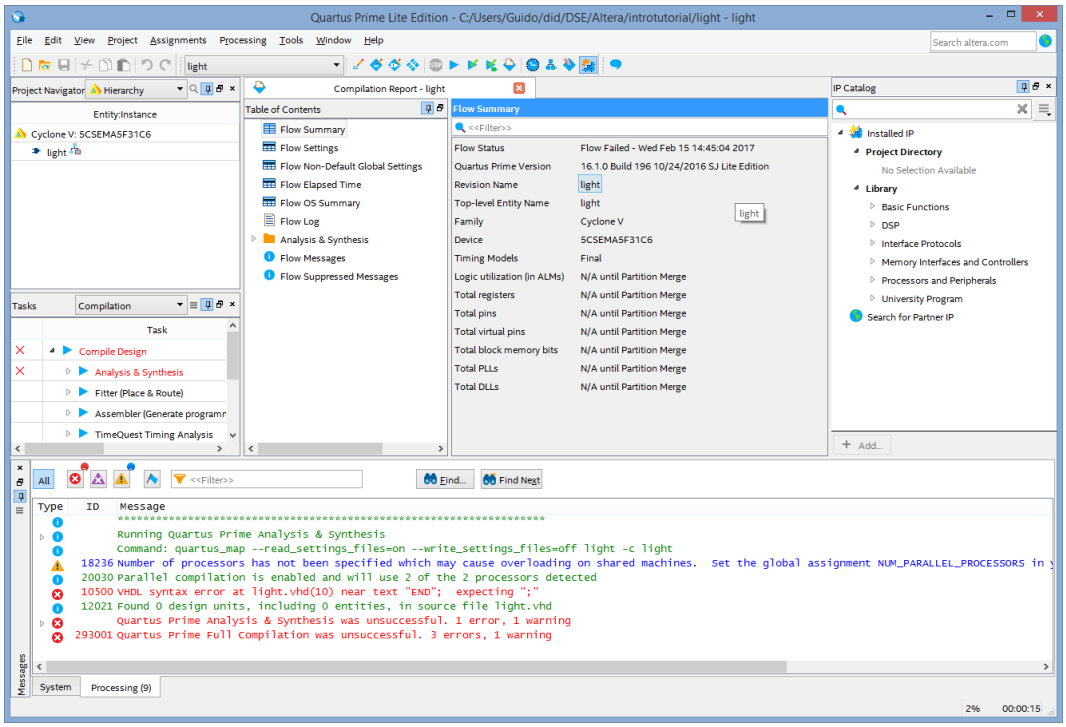


Figure 18 – Error messages and compilation report for the failed design.

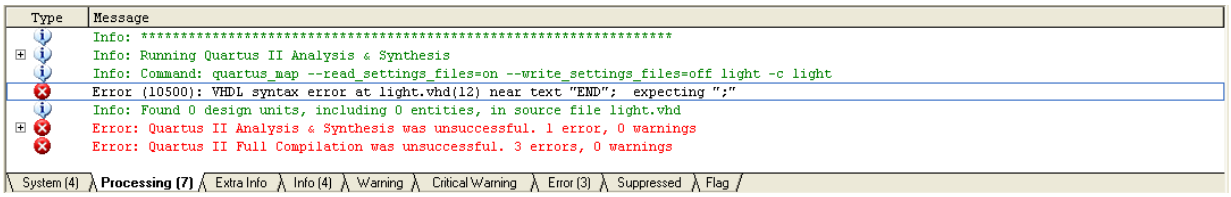


Figure 19 – Compilation errors.

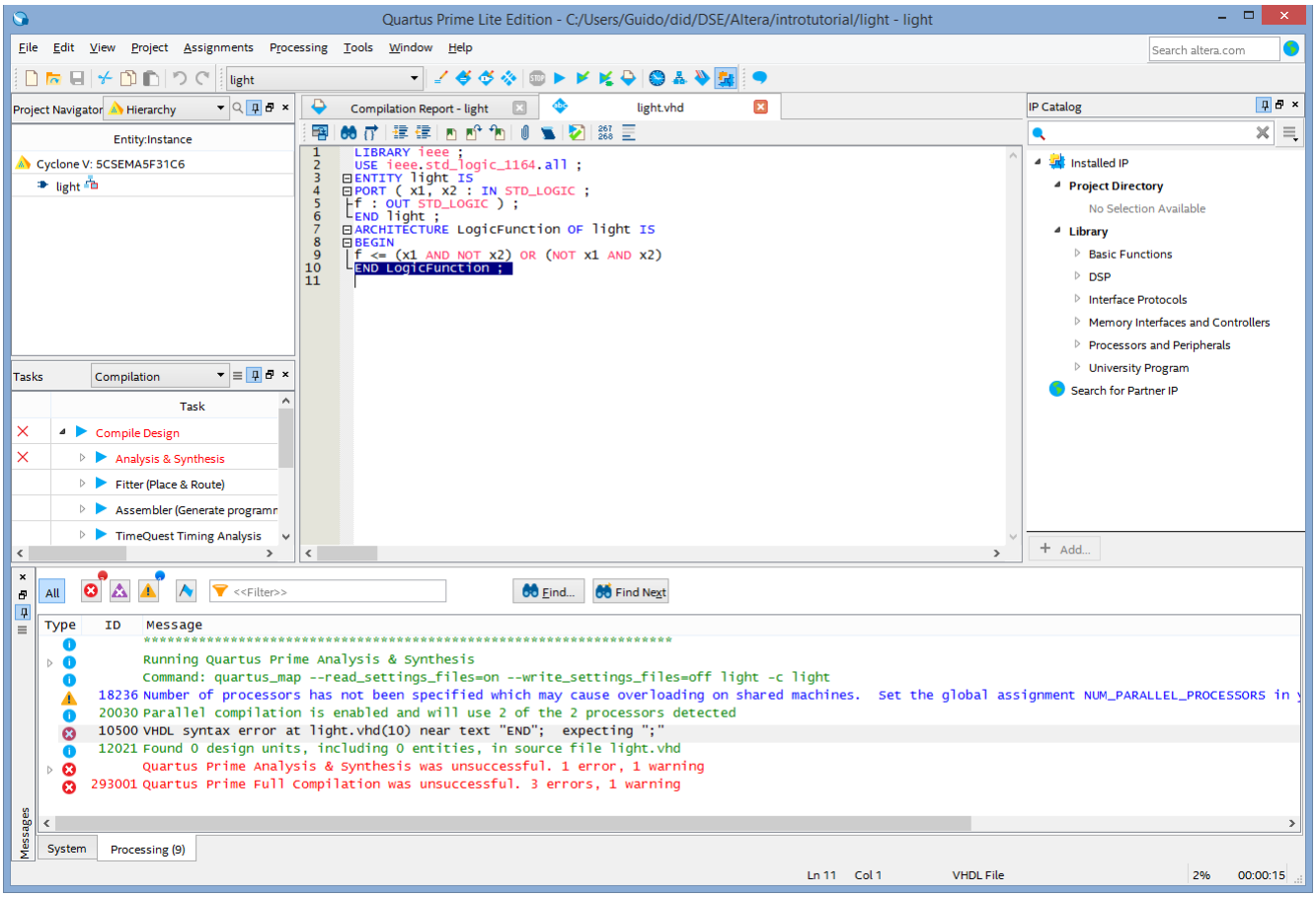


Figure 20 - Identifying the location of the error.

5 - Pin Assignment

During the above compilation, the Quartus Prime Compiler was unconstrained, thus the tool could choose any pins on the FPGA as inputs or outputs. However, the DE1-SoC board has hardwired connections between the FPGA pins and the other components on the board. We will use two toggle switches, labeled SW0 and SW1, to provide the external inputs, x1 and x2, to our example circuit. These switches are connected to the FPGA pins PIN_AB12 and PIN_AC12, respectively. We will connect the output `f` to the red light-emitting diode labeled LEDR0, which is hardwired to the FPGA PIN_V16.

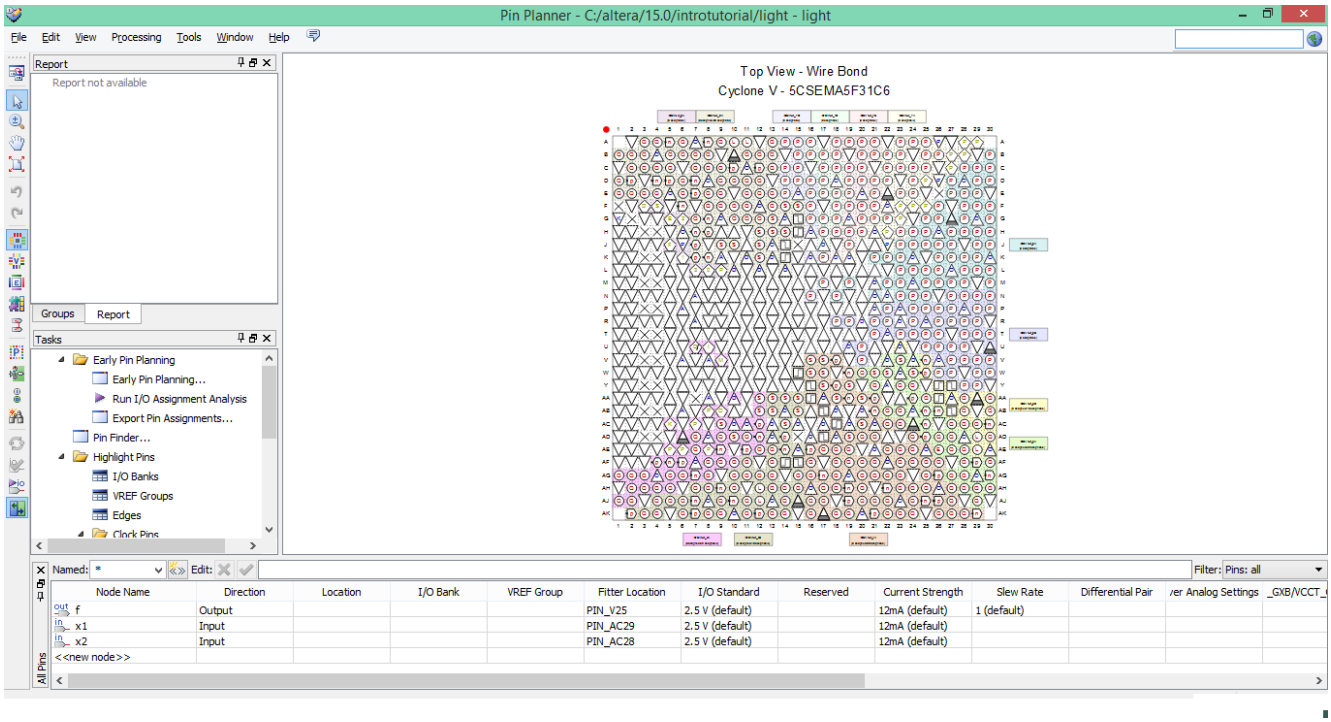


Figure 21 - The Pin Planner window.

Pin assignments can be done by using the Assignment Editor. Select Assignments > Pin Planner. This command invokes a sub-part of Quartus Prime which is called Pin Planner. Window in Figure 21 should appear.

This editor shows also the pin-out of the Cyclone V FPGA for identifying easily the mapped pins once configured. As shown in the figure, it is necessary to assign a location to each pin of the design that is `f`, x1 and x2. Double-click on the entry <<Location>> corresponding to the pin x1. The drop-down menu in Figure 22 will appear. Click on x1, the first pin to be assigned; this will enter x1 in the displayed table. Double-click on the box to the right of this new x1 entry, in the column labeled Location. Scroll down and select PIN_AB12. The item in Figure 23 will appear. Instead of scrolling down the menu to find the desired pin, you can just type the name of the pin (AB12) in the Location box. Use the same procedure to assign input x2 to pin PIN_AC12 and output f to pin PIN_V16. The Pin Planner window with full pin assignment will be the one given in Figure 24. You do not normally need to save the pin assignment since the Pin Planner does it automatically every time a new pin is configured. To save the assignments made, choose File -> Save. Leave the main Pin Planner window open. Now, **recompile the circuit, so that it will be routed with the correct pin assignments. Note that you must recompile the source code every time you modify the pin assignments because the synthesizer has to select the correct logic elements inside the FPGA and the place & route software has to route them to your selected pins.**

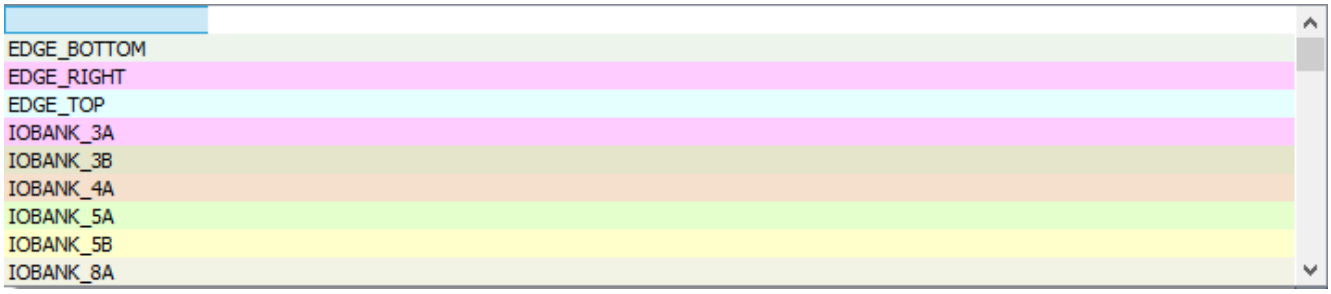


Figure 22 - The drop-down menu that views the location names.

PIN_AB12	IOBANK_3A	Column I/O	DIFFIO_RX_B11n, DIFFOUT_B11n
PIN_AB13	IOBANK_3B	Column I/O	DIFFIO_RX_B19n, DIFFOUT_B19n, DQSn3B
PIN_AB15	IOBANK_3B	Column I/O	DIFFIO_RX_B27p, DIFFOUT_B27p, DQS4B
PIN_AB17	IOBANK_4A	Column I/O	CLK2n, DIFFIO_RX_B47n, DIFFOUT_B47n
PIN_AB21	IOBANK_4A	Column I/O	DIFFIO_RX_B79n, DIFFOUT_B79n
PIN_AB22	IOBANK_5A	Row I/O	DIFFIO_RX_R8p, DIFFOUT_R8p, DQ1R
PIN_AB23	IOBANK_5A	Row I/O	DIFFIO_RX_R8n, DIFFOUT_R8n, DQ1R
PIN_AB25	IOBANK_5A	Row I/O	DIFFIO_RX_R9n, DIFFOUT_R9n
PIN_AB26	IOBANK_5A	Row I/O	DIFFIO_TX_R7n, DIFFOUT_R7n
PIN_AB27	IOBANK_5B	Row I/O	CLK5n, DIFFIO_RX_R21n, DIFFOUT_R21n

Figure 23 - The selected pin AB12.

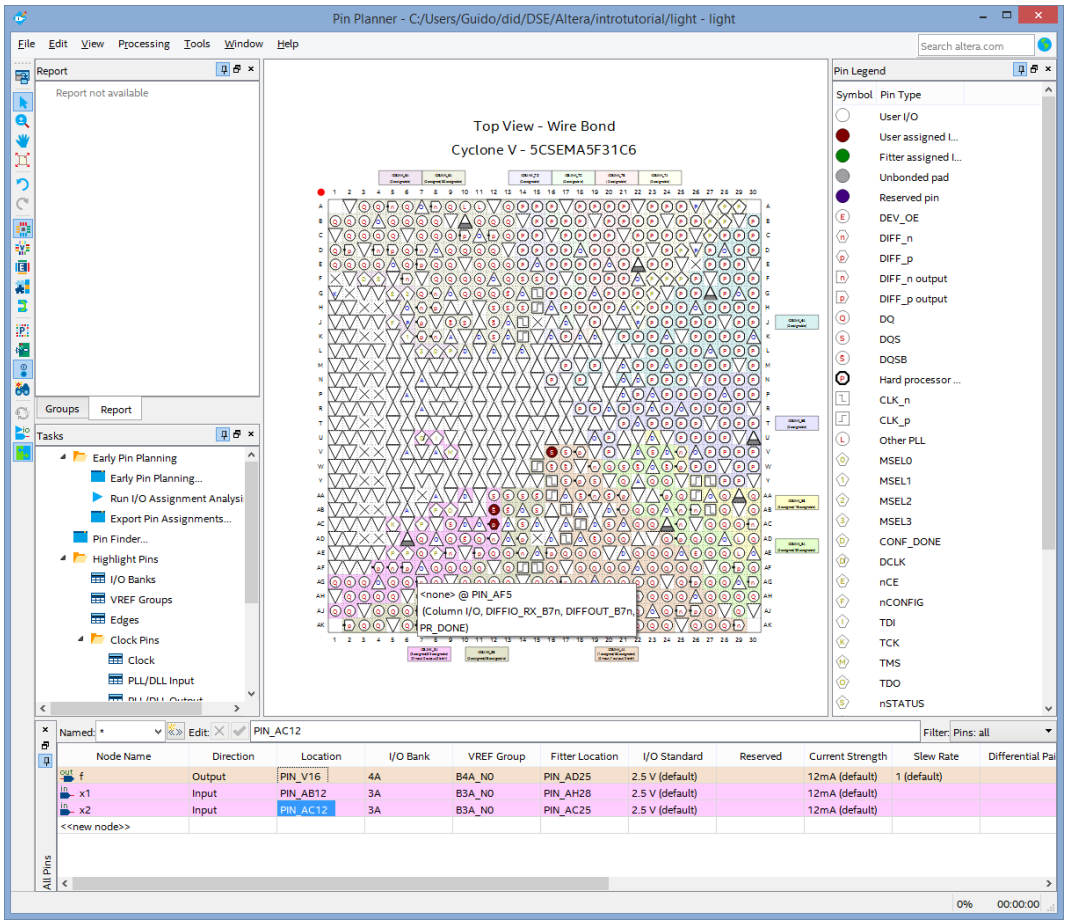


Figure 24 - The Pin Planner with the complete assignment.

As the DE1-SoC board has fixed pin assignments, the user will probably want to use the same pin assignment for subsequent designs. Going through the procedure described above becomes tedious if the project requires the assignment of many pins. Quartus Prime has a feature which allows the user to both export and import the pin assignments from a special file format, rather than creating them manually using the Assignment Editor. A simple file format can be used for this purpose: it is the *quartus settings file (QSF)* one, which is a common text file format that contains comma-delimited values. This file format is often used in conjunction with Microsoft Excel or other spreadsheet tools, but the file can also be created by hand by using any plain ASCII text editor. The format of the file for our simple project is

```
To, Location
x1, PIN_AB12
```

```
x2, PIN_AC12
 f, PIN_V16
```

If you open the file with Notepad for example, you will find a lot of information regarding not only the pin and their direction, but also the I/O levels and the groups. This information will be very valuable in the next classes when you will be asked to design complete systems with a relevant numbers of I/O pins. In these cases, pin plan is fundamental to guarantee design constraints such as speed and area usage or just to connect the FPGA to external chips with the correct logic levels. Of course, all the data contained in this file is available in graphic format in the Pin Planner window.
By adding lines to the file, any number of pin assignments can be created. Such *qsf* file can be imported into a design project. If you created a pin assignment for a particular project, you can export it and use it in a different project. To see how this can be done, open again the Assignment. Now, select Export Assignment. Figure 25 should appear.

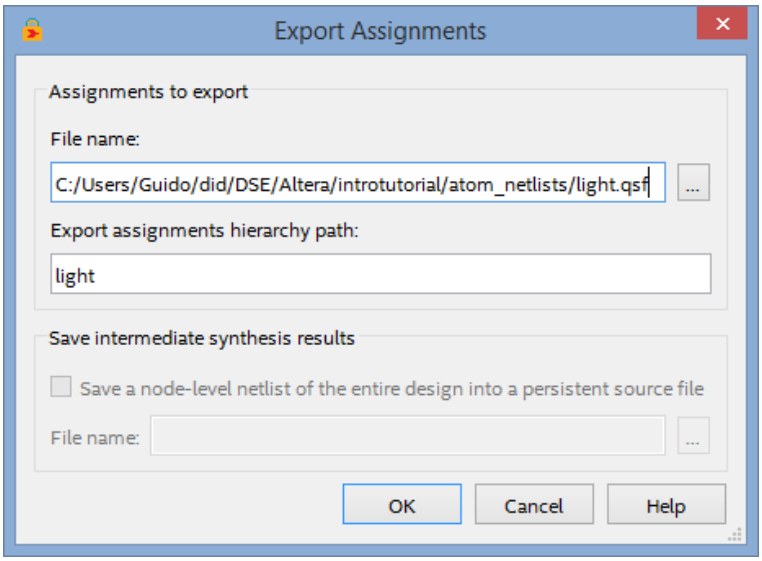


Figure 25 - Exporting the pin assignment.

You can import a pin assignment by choosing Assignments > Import Assignments in the main Quartus Prime window. This action opens the dialogue window of Figure 26 that allows the user to select the file to import. Type the name of the file, including the *csv* extension and the full path to the directory that includes the file in the File Name: box and press OK. Of course, you can also browse and find another file if required.

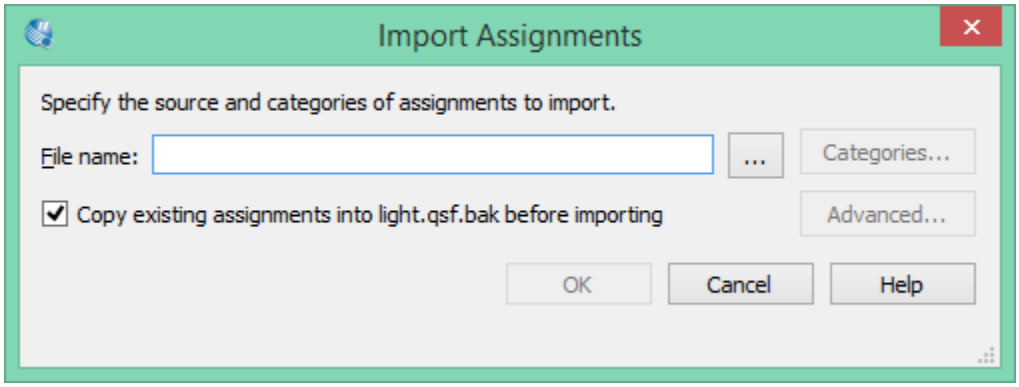


Figure 26 - Importing the pin assignment.

In large designs, for convenience all relevant pin assignments of the DE1-SoC board are given in the file called DE1_SoC.qsf in the web site of the course. This file uses the names found in the *DE1-SoC User Manual* (also available on the web pages). **If we wanted to assign the pins for our circuit by importing this file, then we would have to use the same names in our HDL design file; namely, *SW[0]*, *SW[1]* and *LEDR[0]* for *x1*, *x2* and *f*, respectively. For example, in the DE1_Soc.qsf file the 10 toggle switches are called *SW[9]* to *SW[0]*; since VHDL uses parentheses rather than square brackets, these switches are referred to as *SW(9)* to *SW(0)* in VHDL files. They can also be referred to as an array, such as *SW(9 downto 0)*.**

LABORATORY: read and study the following part, it is the basis for next lab sessions. Be aware that to complete this part you need the DE1-SoC board connected to your PC

6 – Simulating the Designed Circuit

Before implementing the designed circuit in the FPGA chip on the DE-series board, it is prudent to simulate it to ascertain its correctness. The suggested procedure is to exploit an external HDL simulation environment, like for example Modelsim or Questasim.

In a functional simulation, it is assumed that logic elements and interconnection wires in the FPGA are perfect, thus causing no delay in propagation of signals through the circuit. A more complex alternative is to take all propagation delays into account, which leads to timing simulation. Typically, functional simulation is used to verify the functional correctness of a circuit as it is being designed. Having ascertained that the designed circuit is functionally correct, we should now perform the timing simulation to see how it will behave when it is actually implemented in the chosen FPGA device. Unlike functional simulations, timing simulations require the full compilation of your design, not just Analysis and Synthesis. However, timing simulations are only supported for a limited subset of FPGAs, not including the Cyclone V.

Follow the guidelines at the end of this document to perform a complete simulation of the circuit and to assess its behavior.

7 – Programming and Configuring the FPGA

The FPGA device must be programmed and configured to implement the designed circuit. The required configuration file is generated by the Quartus Prime Compiler's Assembler module. The DE1-SoC board allows two different configuration ways, known as JTAG and AS modes. The configuration data is transferred from the host computer (which runs the Quartus Prime software) to the board by means of a cord that connects a USB port on the host computer to the leftmost USB connector on the board. To use this connection, it is necessary to have the USB-Blaster driver installed. If this driver is not already installed, consult the DE1-SoC *Getting Started* manual for information about installing the driver. Before using the board, make sure that the USB cord is properly connected and turn on the power supply switch on the board.

In the JTAG mode, the configuration data is loaded directly into the FPGA device. The acronym JTAG stands for Joint Test Action Group. Originally this interconnection has been defined by a group that created a simple connector for testing digital circuits and loading data into them. This became an IEEE standard during the past years. If the FPGA is configured in this way, it will hold its configuration as long as the power remains turned on. The configuration information is lost when the power is turned off.

The second possibility is to use the Active Serial (AS) mode. In this case, a configuration device that includes some flash memory (EPCQ256) is used to store the configuration data. Quartus Prime dumps the configuration data in the configuration device on the DE1-SoC board. Then, the data is loaded into the FPGA upon power-up or reconfiguration. The FPGA does not need to be configured by the Quartus Prime software if the power is turned off and on.

7.1 - JTAG Programming

The programming and configuration task is performed as follows. Select Tools > Programmer to reach the window in Figure 27. Here it is necessary to specify the programming hardware and the mode that should be used. If not already chosen by default, select JTAG in the Mode box. Also, if the DE1-SoC[USB-1] is not chosen by default, press the Hardware Setup... button and select the DE1-SoC[USB-1] in the window that pops up, as shown in Figure 28.

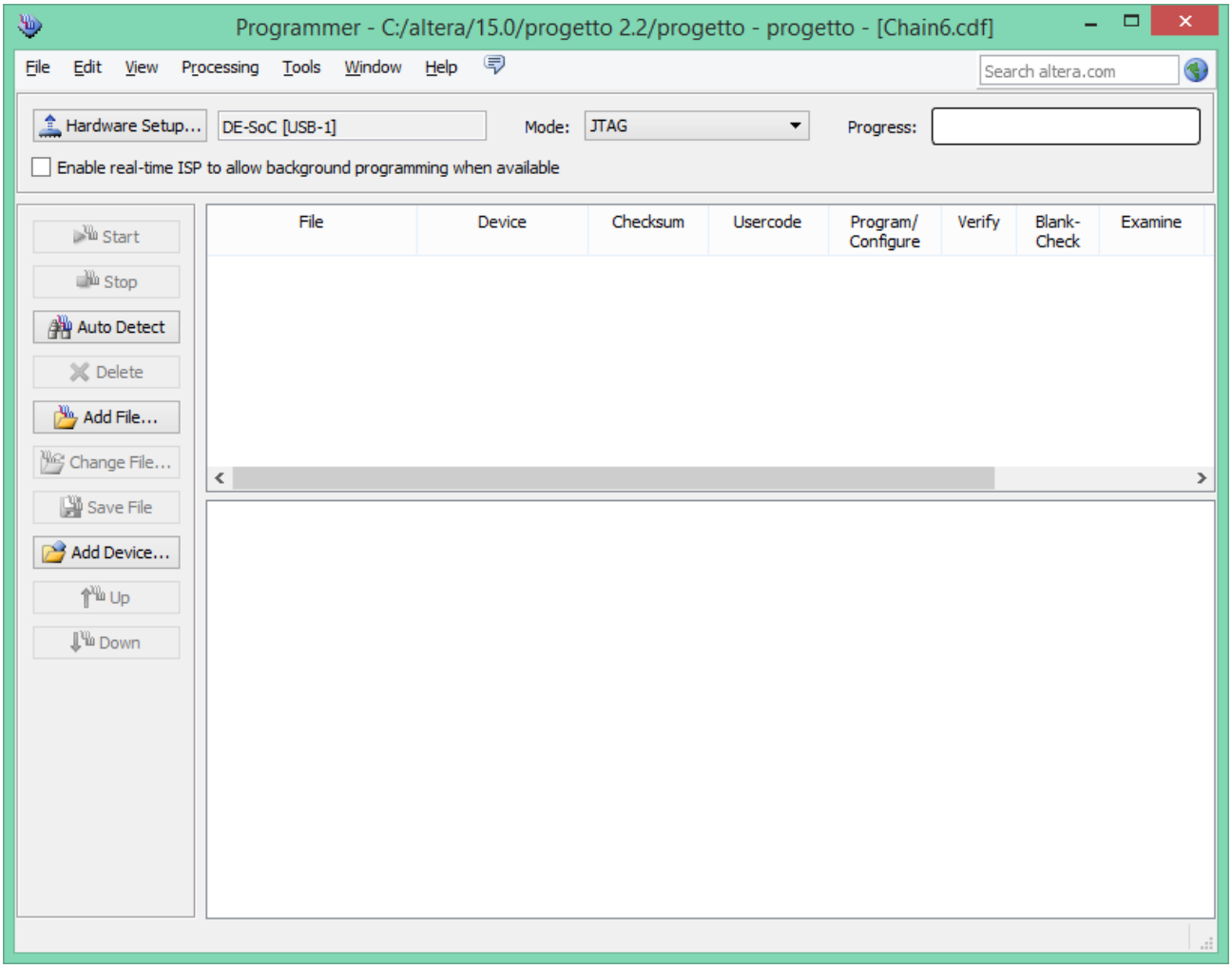


Figure 27 - The Programmer window.

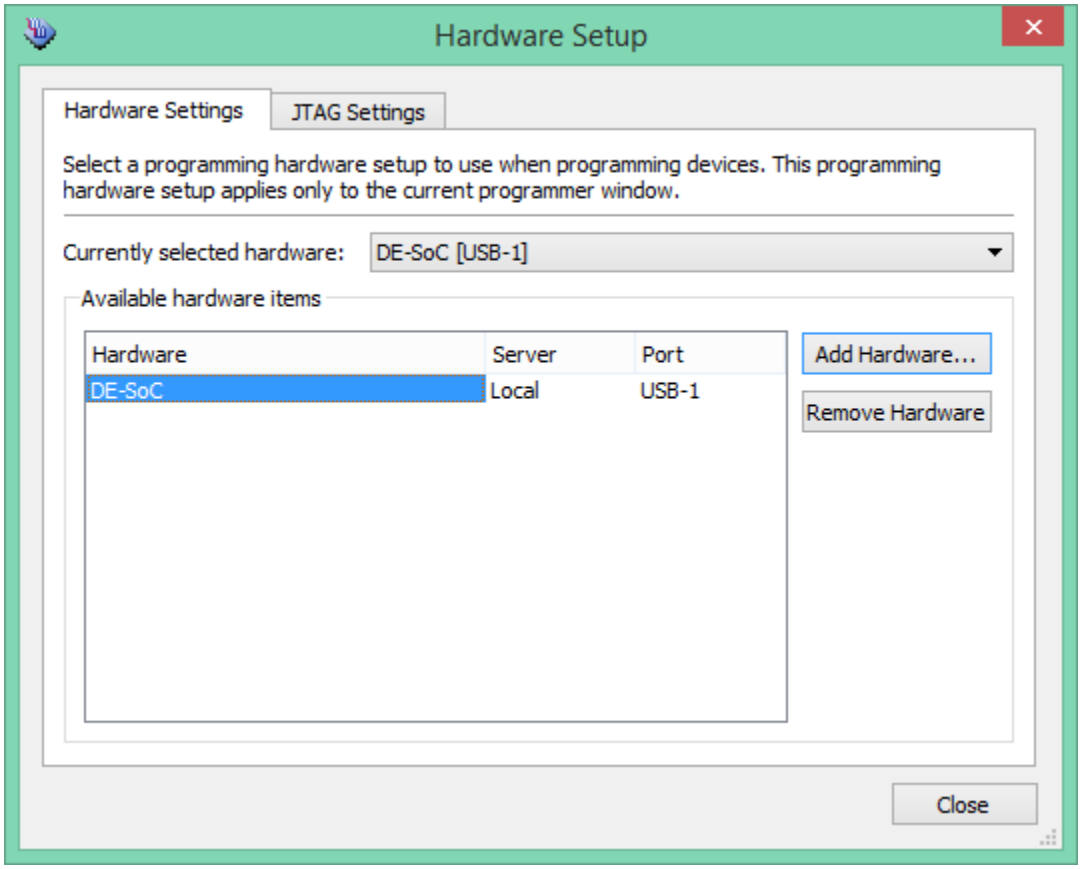


Figure 28 - The Hardware Setup window.

The next step is to click Auto Detect, this command will open a window in which it is possible to select the device, as shown in the figure 29. Now you can choose the correct device, which is 5CSEMA5.



Figure 29 - Select Device window.

As it can be seen in the figure 30, the updated Programmer windows shows two devices connected in series, the first one is the Hardprocessor, the second one is the FPGA. Now you have to click the FPGA device (5CSEMA5) and then right click mouse to popup the manual, so you can select Change File. At this point you have to select the directory named "output_files" and then select the file with the extension *.sof,* which stands for SRAM Object File. The figure 31 shows the selected directory. Click on the Program/Configure check box as shown in Figure 32.

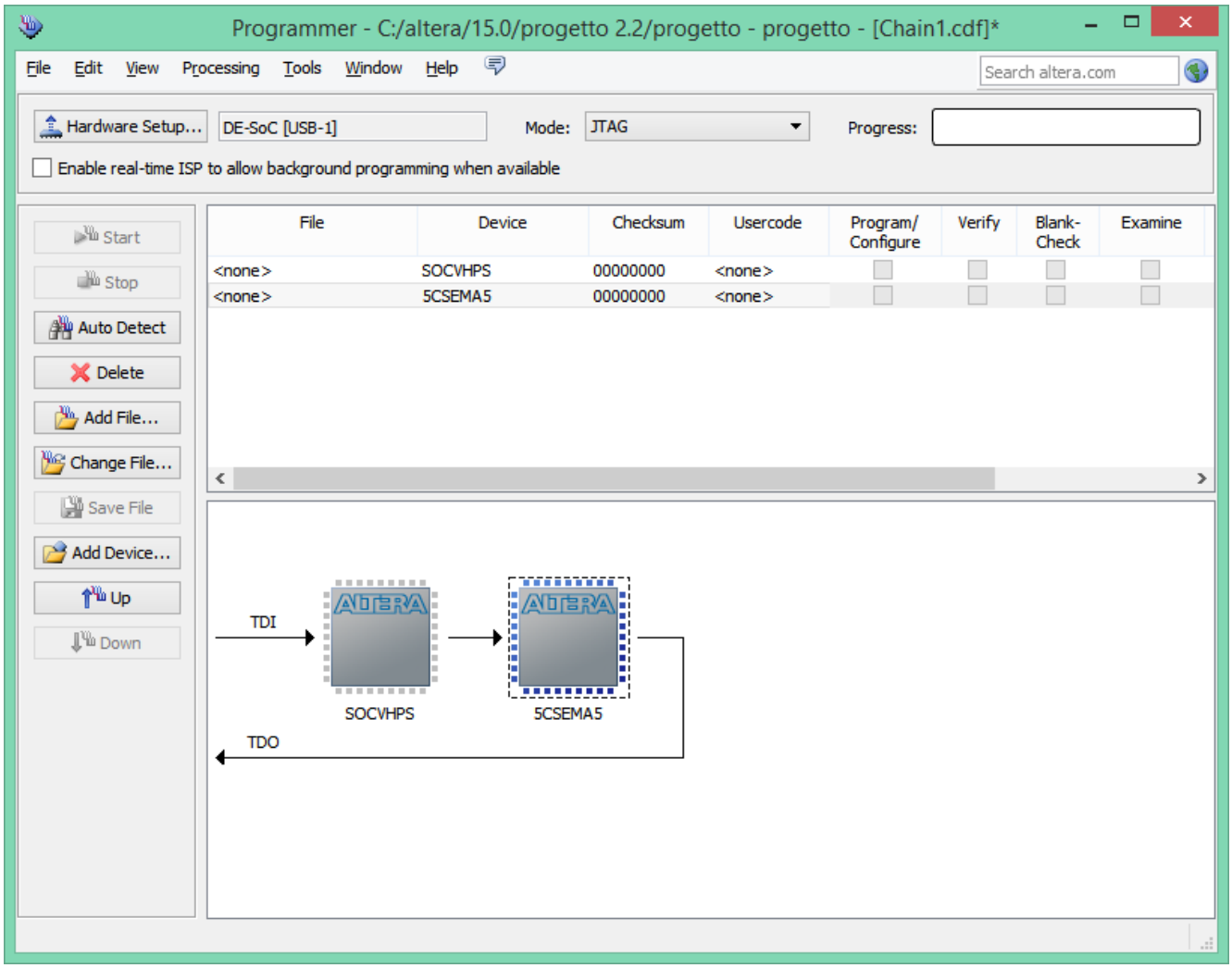


Figure 30 - The updated Programmer window.

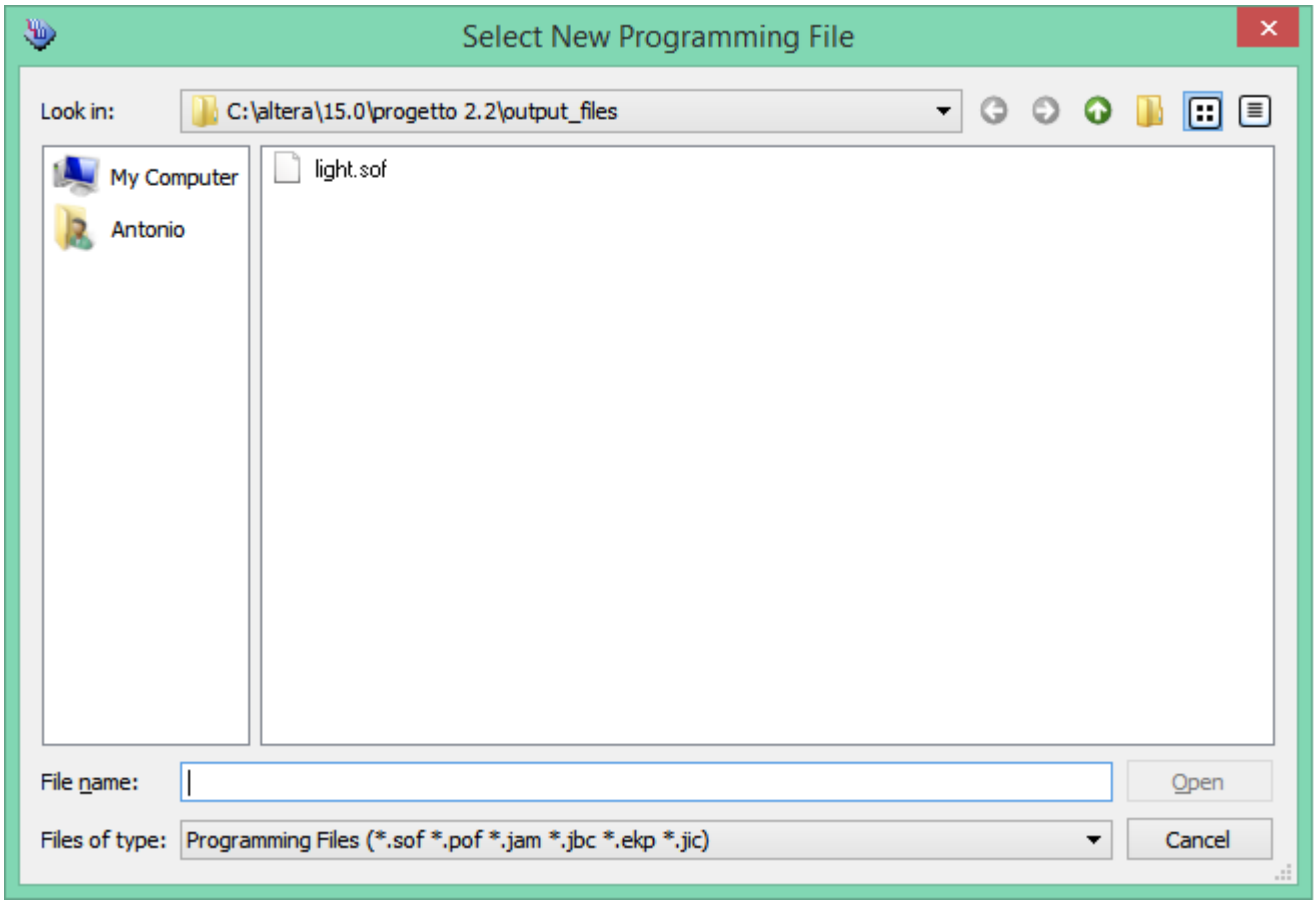


Figure 31 – Output files directory window.

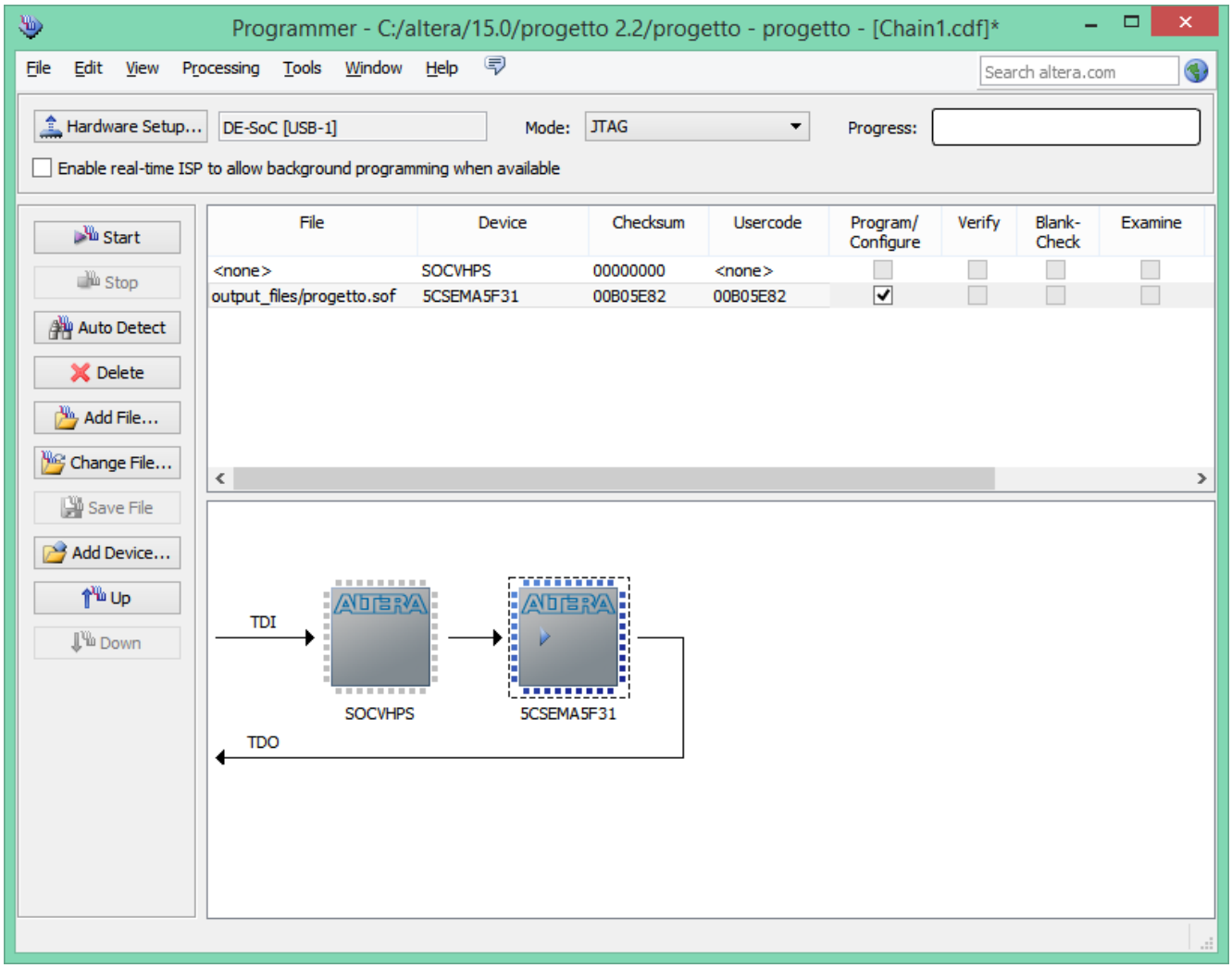


Figure 32 - The updated Programmer window.

Now press Start in the window shown in Figure 32. A LED on the board will light up when the configuration data has been downloaded successfully. If you see an error reported by Quartus Prime indicating that programming failed check the power supply connection on the board and ensure that it is properly powered on.

8 - Testing the Designed Circuit

After having downloaded the configuration data in the FPGA device, you can now physically test the circuit. Try all four valuations of the input variables x1 and x2, by setting the corresponding states of the switches SW_1 and SW_0. Verify that the circuit implements the truth table of Figure 12.

If you want to make changes in the design you can try. First close the Programmer window. Then make the changes in the HDL files, compile the source file and program the board as explained previously.

Use of ModelSim/QuestaSim

A digital design is usually composed of many components, each of which has to be tested in isolation and then integrated into a design when it operates correctly. Once each component has been verified, the designer has to test the whole system.
To verify that both single components and the complete system operate correctly, we use simulation, which is a process of testing the design by applying inputs (test vectors) to the circuit and observing its behaviour. The output of a simulation is a set of waveforms that show how a circuit behaves based on the applied sequence of inputs. If necessary, internal signals can be observed too.

If we model the circuit by means of an HDL, then an HDL simulator must be used to run simulations. This short tutorial introduces HDL simulation using ModelSim/QuestaSim, where input sequences are generated by using an HDL to create dedicated code able to generate all sequences required to test the circuit. This code essentially tells the simulator what values to set the inputs to, and what outputs are expected for those inputs.

Note: despite QuestaSim Altera Starter Edition is free, you have to download a license from the Altera web-site (https://www.altera.com/SSLC). Then, you have to create an environment variable named LM_LICENSE_FILE and set it to point to the license file.

The concept of testbench

Using an HDL to stimulate the design highlights that the simulated HDL code is always composed of two parts:

1. The inner code, named device under test (DUT) or unit under test (UUT), which models the designed component or systems that we want to test and verify. This code can be as simple as a single gate, described in a single file, or it can be a hierarchy of HDL models, which describe a large and complex system with a mixed structural and behavioural style.
2. The outer code, named as testbench, which is responsible for the generation of input signals. These signals are created according to a given algorithm or sequence of HDL statements, and applied to the input ports of the DUT. The testbench might also include statements to verify that the outputs returned by the DUT are correct. An alternative approach to the test of the DUT relies on read data from file and write data to file, by relying on proper HDL statements. In this approach, we need a separate model of the DUT, usually a pure functional model, developed using a high-level programming language, such as Matlab, C, Python, …. The behaviour of this software model is independently verified and it is used as golden or reference model. Both the software model and the DUT read the same sequences of inputs from an external data file and save the generated outputs into two separate files. These output files are then compared to see if any error is detected.

In other words, the testbench is a sort of wrapper that encapsulates the DUT in a unique HDL simulation model. While the synthesis is applied only to the DUT model, the simulation must be run on the whole system formed by testbench and DUT. The testbench is not intended to be synthesized, therefore, it can exploit all features of HDLs, including non-synthesizable data type and statements. Some of the advantages offered by the testbench approach are:

- Input and Output test vectors can be easily documented
- More methodical approach
- More automated testing
- Same functionality tests can be repeated easily
- Same test bench can be used for verifying both the original HDL source code and timing-enhanced synthesized code described by a post fit model

Running ModelSim/QuestaSim

To start Modelsim/QuestaSim we need to run the main executable file, which can be directly linked to the operating system as a shortcut or can be launched/invoked from the installation folder (from the Quartus Prime installation folder locate *questa_fse/win64* and run `vsim.exe`). After starting the ModelSim/QuestaSim program (FPGA Starter Edition), we get the window in Figure 1, which consists of three sections:

- the main menu at the top,

- a set of workspace tabs (here, only the library tab is shown),
- a command prompt at the bottom.

The menu is used to access functions available in ModelSim/QuestaSim. The workspace contains a list of modules/entities and libraries of modules/entities available to you, as well as details of the project you are working on. A new work area will appear on the right of the libraries of modules/entities when needed to display waveforms and/or text files. Finally, the command prompt at the bottom shows feedback from the simulation tool and allows users to enter commands.
We begin by creating a project where all design files to be tested are included as well as the testbench and then we compile the HDL files included in the project. Finally, we select the signals we want to display as waveforms and run the simulation. Based on the results of the simulation, the design can be altered until it meets the desired specifications.

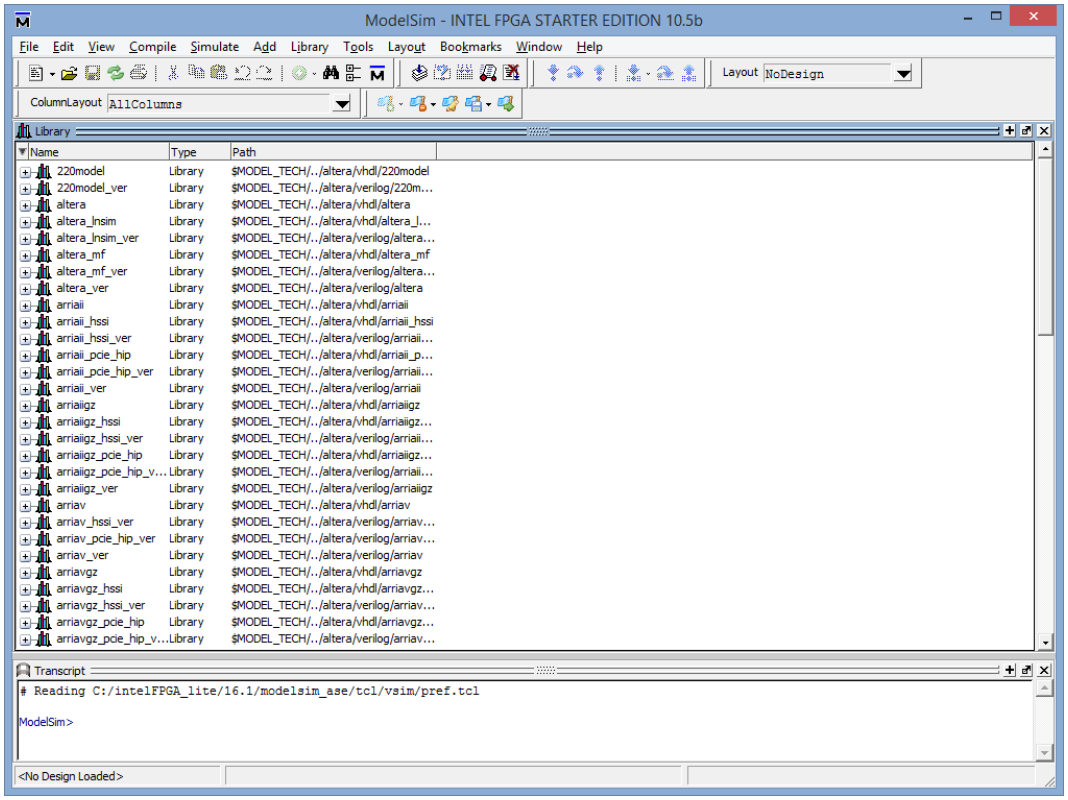


Figure 1: Initial ModelSim window.

Creating a project

To create a project in ModelSim/QuestaSim, select **`File > New > Project`**.... A *Create Project* window shown in Figure 2 will appear.

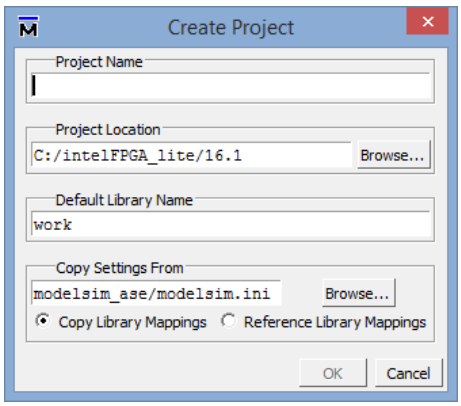


Figure 2: Creating a new project.

The Create Project window consists of several fields: project name, project location, default library name, and copy settings field.
Project name is a user-selected name and the location is the directory where the source files are located. We choose the project name to be **`light`**, to match the top-level entity name of the example design used in this tutorial, and the location of the project is a subdirectory.
The default library name field specifies a name by which ModelSim/QuestaSim catalogues designs. For example, a set of files that describe the logical behaviour of components in Cyclone V devices is available. This allows the simulator to include a set of files in simulation as libraries rather than individual files. For the purpose of this tutorial, you can use *work* as the library name for your project.
The last field in the create project window is the copy settings field. This allows default settings to be copied from the initialization file and applied to your project. Now, click OK to proceed to add files to the project using the window shown in Figure 3.

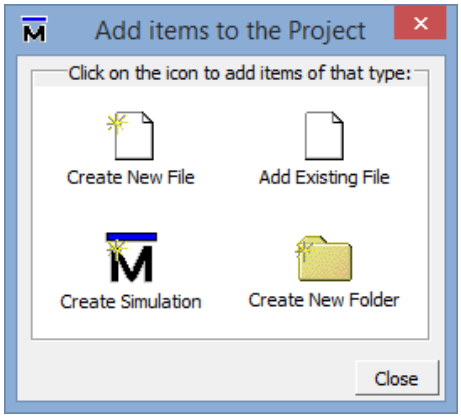


Figure 3: Adding items to a project.

This window gives several options to add files to the project, including creating new files and directories, or adding existing files. Since the file for this tutorial exists, click Add Existing File and select **`light.v/light.vhd`** file. Once the file is added to the project, it will appear in the Project tab on the left-hand side of the screen (Figure 4).

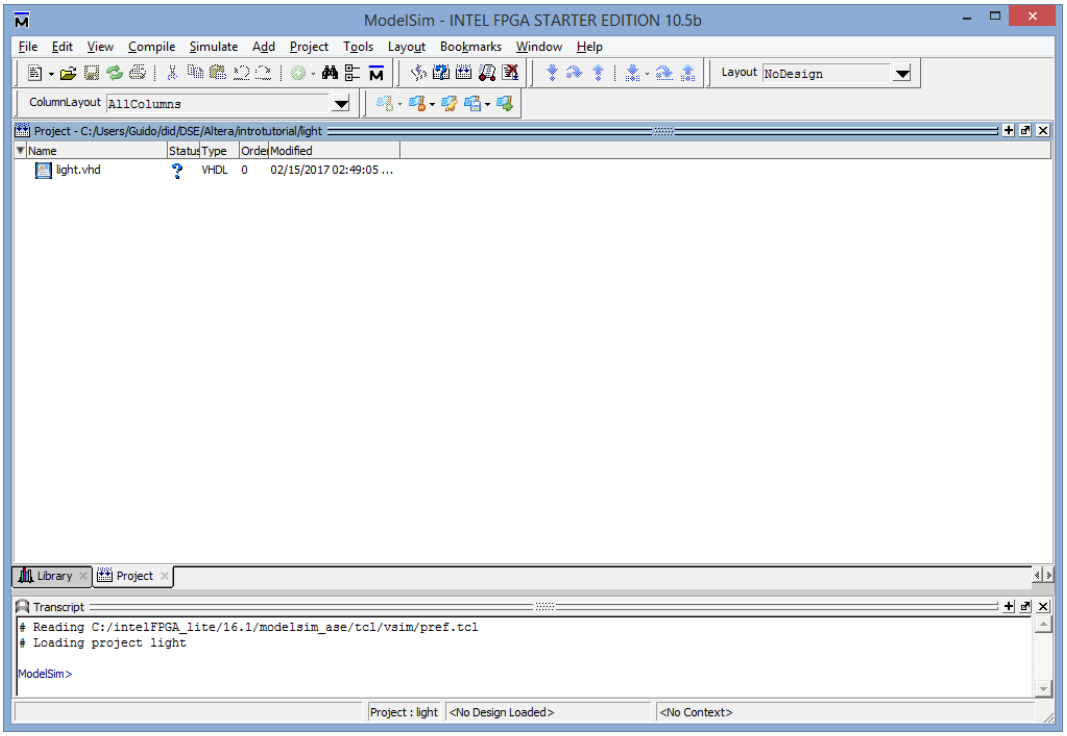


Figure 4: Workspace window after the project is created.

Adding a testbench

To stimulate the DUT we can rely on a testbench approach. To do so, we have to create a new module/entity, which includes the DUT as a component and all the HDL statements required to generate the required stimulation.
The module/entity has no input/output ports and therefore it only consists of the two following lines:

```
// Verilog
module light_tb;

-- VHDL
ENTITY light_tb IS
END light_tb;
```

where **light_tb** is the name assigned to the testbench unit. For our example, one possible implementation for the testbench is shown below:

```
// Verilog
`timescale 1ns / 1ps
```

```
module light_tb;

    reg x1_tb;
    reg x2_tb;
    wire f_tb;

    light uut (
        .x1(x1_tb),
        .x2(x2_tb),
        .f(f_tb)
    );

    initial begin
        x1_tb = 1'b0;
        x2_tb = 1'b0;
        #200;

        x2_tb = 1'b1;
        #200;

        x1_tb = 1'b1;
        x2_tb = 1'b0;
        #200;

        x2_tb = 1'b1;
    end

endmodule

-- VHDL
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY light_tb IS
END light_tb;

ARCHITECTURE mytest OF light_tb IS

   SIGNAL x1_tb, x2_tb, f_tb : STD_LOGIC;

   COMPONENT light
      PORT ( x1, x2 : IN STD_LOGIC ;
                   f : OUT STD_LOGIC ) ;
   END COMPONENT;

BEGIN

   uut : light PORT MAP (x1 => x1_tb, x2 => x2_tb, f => f_tb);
```

```
    PROCESS
    BEGIN
      x1_tb <= '0';
      x2_tb <= '0';
      WAIT FOR 200 ns;
      x2_tb <= '1';
      WAIT FOR 200 ns;
      x1_tb <= '1';
      x2_tb <= '0';
      WAIT FOR 200 ns;
      x2_tb <= '1';
      WAIT;
    END PROCESS;

END mytest;
```

In the testbench architecture, we define a signal for each input or output port of the DUT and these signals are mapped to the IO ports. The waveforms are generated using a special statements such as the *initial* environment in Verilog and the *process* environment (with no sensitivity list) in VHDL, where each transition is explicitly created by means of an assignment. The delays required between transitions are introduced by placing proper **#N** or **`wait`** statements. Since in the Verilog description after updating *x2_tb* no other changes are applied, the values are then kept fixed. Similarly, in VHDL the *process* has an empty sensitivity list and it is closed by a wait statement with no delay, the process is executed a single time and then frozen. Note that in Verilog we need a special statement (*timescale*) to specify the scale of the delays applied in the model.

To use this testbench in the simulation, follow this sequence of steps:

1. With **File -> New -> Source -> Verilog/VHDL**, open a text editor window and enter the code above (Verilog or VHDL)
2. Save the file with name `light_tb.v/light_tb.vhd`
3. With **Project -> Add to Project -> Existing File**, open the Add File to project window shown in Figure 5
4. Browse the tutorial folder and select the created file, then click OK. The testbench file is now included in the project window, together with the `light.v/light.vhd` file.
5. Compile both files using **Compile -> Compile All**

   When the project includes multiple files, compilation order is important. Follow these steps to change the compilation order within a project.
   - Select **Compile > Compile Order**.
     This opens the *Compile Order* dialog box.
   - Click the Auto Generate button.
     ModelSim/QuestaSim "determines" the compile order by making multiple passes over the files. It starts compiling from the top; if a file fails to compile due to dependencies, it moves that file to the bottom and then recompiles it after compiling the rest of the files. It continues in this manner until all files compile successfully or until a file(s) cannot be compiled for reasons other than dependency.

6. Now, start again the simulation, using **`Simulate -> Start Simulation.`** In the start Simulation window, expand the *work* item and select `light_tb`, which is the name of the module/entity contained in the testbench file. Note: check that when the simulation starts, the optimization options are giving visibility to the all design - Apply full visibility to all modules (full debug mode), this corresponds to invoke *vsim* with the option *-voptargs=+acc*

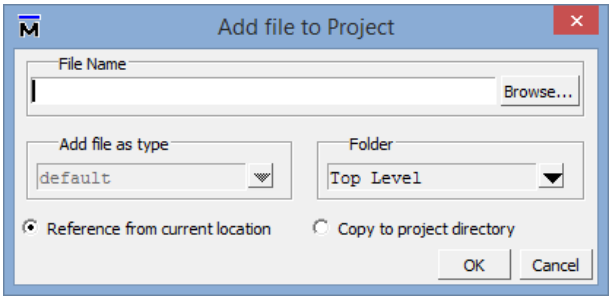


Figure 5: Add File to Project window.

7. The window in Figure 6 should appear.

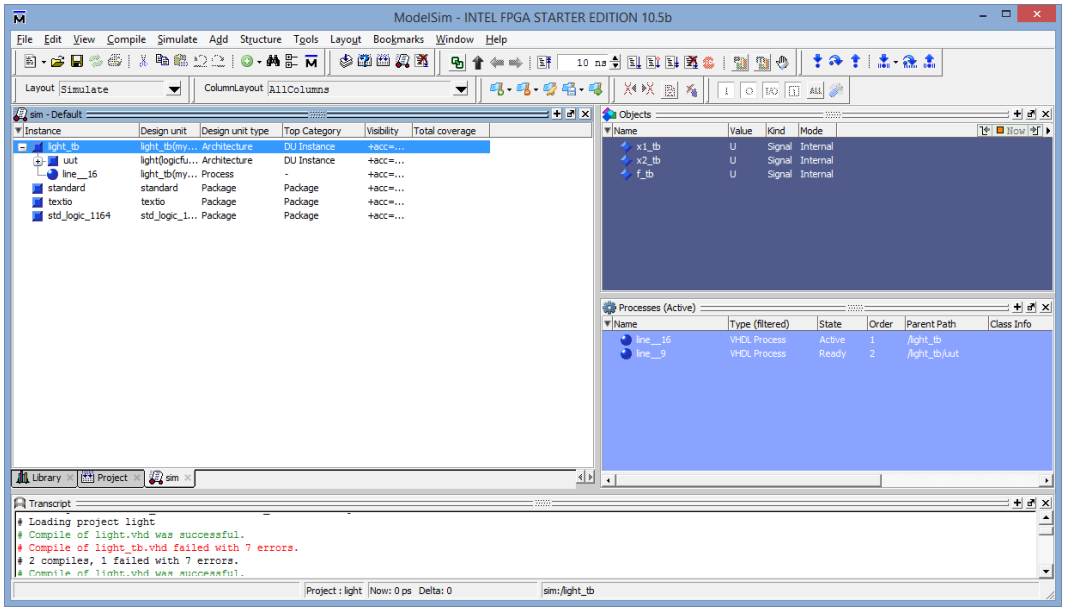


Figure 6: Simulation window.

You will see `light_tb` as the top-level module/entity of the simulation hierarchy and the encapsulated component `uut` at the second level. This is shown close to the top left corner of the window. Right click on the `light_tb` module/entity name and follow **`Add to -> Wave -> all items in design`**. This command has the effect of importing all signals declared in the design (both testbench and encapsulated DUT) in the Wave window (Figure 7).

8. To obtain simulation results, it is now sufficient to enter a proper run length in the Wave window (800 ns in this case) and then click on the Run command. After clicking on the Zoom Full command, the whole set of waveforms appear, like shown in Figure 8.

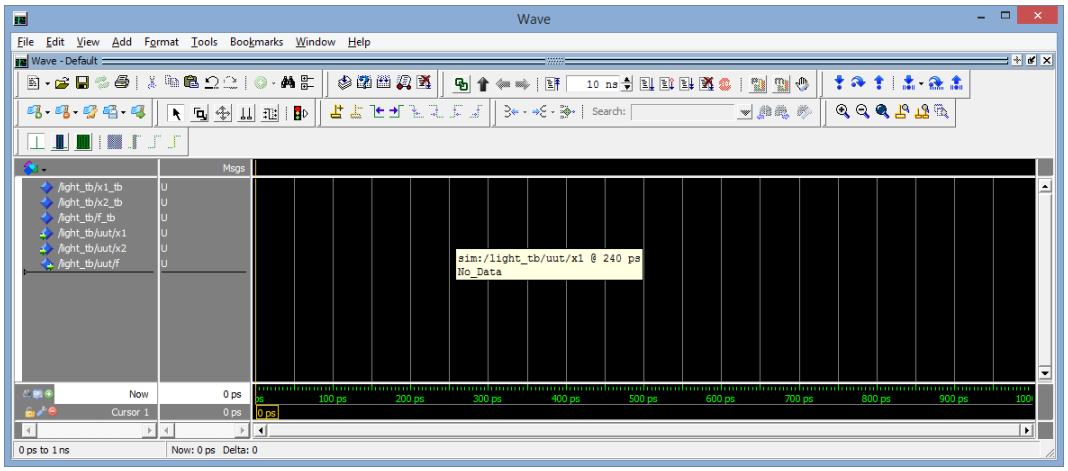


Figure 7: Imported signals.

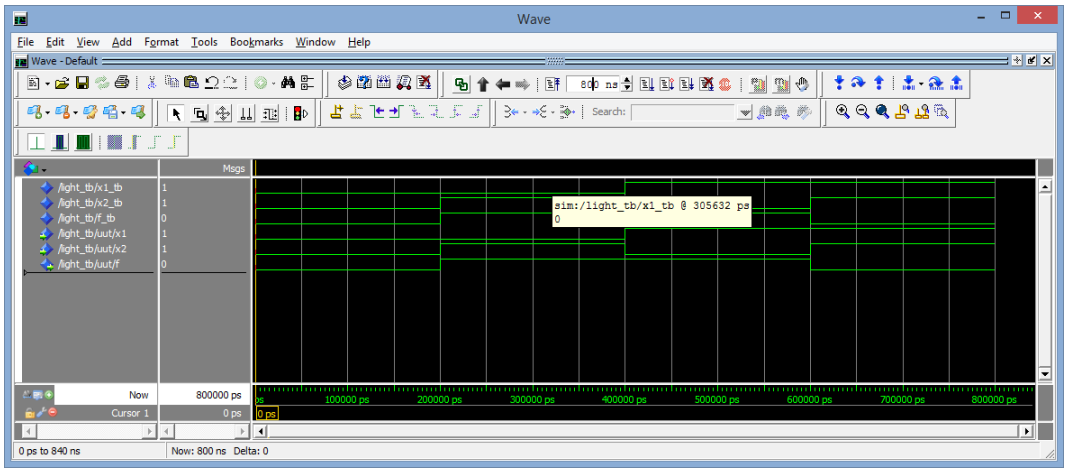


Figure 8: Simulation result.

Several commands in the Wave window allow controlling the shown waveforms and values. For example, by right clicking on the name of a signal, one can change the *Radix* among multiple choices (e.g. binary, octal, hexadecimal, decimal, unsigned, ASCII, …). Another useful feature is given by the buttons located in the top bar of the window and able to move a cursor among edges or events associated with the selected signal.

Although the example used in this tutorial is very simple, HDLs offer many other ways to generate the required input waveforms.

There are three general approaches to the creation of a testbench:

1. Tabular Approach, which uses table of test vectors embedded within the test bench
2. File I/O Approach, which uses table of test vector contained in a separate file
3. Procedural Approach, which uses an algorithm to create inputs, compute expected outputs, and compare with actual outputs.

The tabular approach is the simplest and it tends to be convenient when the DUT can be validated by means of a limited number of input sequences, which can easily generated using explicit assignments.

The file I/O approach is useful when a large number of vectors are needed in source code. In this approach, we list vectors in file and read them into the simulation. Similarly, we record result vectors in a file for later comparison. This method allows you to use same test fixture to run multiple tests accomplished by changing the vector file. File I/O procedure calls are used to read in a line of data from a file, and to write a line of data to a file. Standard procedures are available for characters, strings, bits, booleans, integers, reals. The input files must be located in the working directory, otherwise you have to specify its path in the testbench file.
Finally, the procedural approach is mostly useful when a large number of vectors are used, and an algorithm can easily calculate them.

Several examples for these approaches are shown in the Appendix A.

Appendix A – Examples of testbench

1. Stamentes to generate a clock signal with period 20 ns

- Verilog

```
reg         CLK = 1'b1;

always begin
   #10 CLK = ~CLK;
end
```

- VHDL

```
signal CLK : std_logic := '1';

begin

clock: process
begin
   CLK <= not(CLK);
   wait for 10 ns;
end process clock;
```

2. Statements to generate a reset signal

- Verilog

```
initial begin
   RSTn = 0;
   #15
   RSTn = 1;
end
```

- VHDL

```
reset_gen: process
begin
  rst_n <= '0';
  wait for 15 ns;
  rst_n <= '1';
  wait;
end process reset_gen;
```

3. A testbench to test a four-bit adder/subtracter

- Verilog

```
`timescale 1ns / 1ps

module adder_tb;

  // Inputs
  wire       OPZ;
  reg        CK = 1'b1;
  wire [3:0] A, B;
  wire [4:0] R;

  reg [8:0]  count = 9'd0;
  reg        rst_n = 1'b0;

  // DUT port mapping
  myadder UUT (
    .OPZ(OPZ),
    .A(A),
    .B(B),
    .R(R)
  );

  // Clock generation
  always begin
    #10 CK = ~CK;
  end

  // Reset generation
  initial begin
    rst_n = 0;
    #15
    rst_n = 1;
  end

  // Stimulus generation
  always @(posedge CK or negedge rst_n) begin
    if (!rst_n)
      count <= 9'd0;
    else
      count <= count + 1;
  end

  assign  A   = count[3:0];
  assign  B   = count[7:4];
```

```
  assign  OPZ = count[8];

endmodule
```

- VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder_tb is
end adder_tb;

architecture TB_ARCHITECTURE of adder_tb is

   component myadder
   port(
      OPZ : in std_logic;                    -- with '0' adder,
                                              -- with '1' subtracter
      A,B : in signed(3 downto 0); -- operands
      R : out signed(4 downto 0)   -- result
      );
   end component;

   signal OPZ : std_logic;
   signal CK : std_logic := '1';
   signal A,B : signed(3 downto 0);
   signal R : signed(4 downto 0);
   signal count : unsigned(8 downto 0);
   signal rst_n : std_logic := '0';

   begin
   -- Unit Under Test port map
   UUT : myadder
   port map(
            OPZ => OPZ,
            A => A,
            B => B,
            R => R
            );

   clock: process     -- clock generation
   begin
      wait for 10 ns;
      CK <= not(CK);
   end process clock;
```

```
   reset_gen: process
   begin
     rst_n <= '0';
     wait for 15 ns;
     rst_n <= '1';
     wait;
   end process reset_gen;

   process (rst_n, CK)           -- generation of input vectors
   begin
      if (rst_n = '0') then
         count <= (others => '0');
      elsif (CK'event and CK='1') then
         count <= count + 1;
      end if;
   end process;

   A <= signed(count(3 downto 0));
   B <= signed(count(7 downto 4));
   OPZ <= count(8);

end TB_ARCHITECTURE;
```

4. Generic testbench based on the File I/O approach

- Verilog

```
`timescale 1ns/1ps

module tb_top_design;

  // Parameters
  parameter HALF_PERIOD = 2.5;

  // Signals
  reg clk_tb = 1'b0;
  reg nRST_tb = 1'b0;
  reg [15:0] sample_in_tb;
  wire [15:0] sample_out_tb;

  // DUT instantiation
  top_design DUT (
    .clk(clk_tb),
    .nRST(nRST_tb),
    .sample_in(sample_in_tb),
    .sample_out(sample_out_tb)
  );
```

```
  // Clock generation
  always begin
    #HALF_PERIOD clk_tb = ~clk_tb;
  end

  // Reset generation
  initial begin
    nRST_tb = 0;
    #(2 * HALF_PERIOD);
    nRST_tb = 1;
  end

  // File I/O and input generation
  integer infile;
  integer dataread;
  reg [255:0] line;

  initial begin
    infile = $fopen("data.txt", "r");

    sample_in_tb <= 0;

    wait(nRST_tb == 1);

    while (!$feof(infile)) begin
      @(posedge clk_tb);
      if ($fscanf(infile, "%d\n", dataread) == 1)
        sample_in_tb <= dataread;
    end

    $fclose(infile);
   end

endmodule
```

- VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library std;
use std.textio.all;             -- standard text file i/o
use ieee.std_logic_textio.all;
```

```
entity tb_top_design is
                                  -- empty entity
end tb_top_design;

architecture test of tb_top_design is

  -------------------------------------------------------
  -- Components
  -------------------------------------------------------
  component top_design          -- circuit to be tested
    port (
     clk                        : in  std_logic;
     nRST                       : in  std_logic;
     sample_in                  : in  signed(15 downto 0);
     sample_out                 : out signed(15 downto 0)
    );
  end component;

  -------------------------------------------------------
  -- Constants
  -------------------------------------------------------
  constant half_period : time := 2.5 ns; -- clk period

  -------------------------------------------------------
  -- Internal signals
  -------------------------------------------------------
  signal clk_tb    : std_logic := '0';
  signal nRST_tb   : std_logic := '0';
  signal sample_in_tb     : signed(15 downto 0);
  signal sample_out_tb    : signed(15 downto 0);

begin  -- test

  -- component instantiation
  DUT : top_design
  port map    (
          clk                 =>  clk_tb,
          nRST                =>  nRST_tb,
          sample_in           =>  sample_in_tb,
          sample_out          =>  sample_out_tb
  );


  -- clock generation
  ck_proc : process
  begin
   wait for half_period;
```

```
   clk_tb <= not(clk_tb);
  end process ck_proc;

  -- Reset generation
  reset_gen: process
  begin
    nRST_tb <= '0';
    wait for 2 * half_period;
    nRST_tb <= '1';
    wait;
  end process reset_gen;

  -- waveform generation
  WaveGen_Proc: process(clk_tb, nRST_tb)

       file infile : text open READ_MODE is "data.txt";
       variable  inline     : line;            -- line  declaration
       variable  dataread1  : integer;         -- input data

  begin

   if (nRST_tb = '0') then                     -- reset
      sample_in_tb <= (others => '0');         -- null sample
   elsif (clk_tb'event and clk_tb = '1') then
      if (not endfile(infile)) then
         readline(infile, inline);         -- reading a line
         read(inline, dataread1);          -- reading a value
         sample_in_tb <= to_signed(dataread1, 16);
     end if;
   end if;

  end process WaveGen_Proc;

end test;
```